

Projet réalisé dans le cadre de la présentation au
Titre Professionnel Développeur Web et Web Mobile
(DWWM)

DepanGO

*Plateforme web de mise en relation pour le dépannage
à domicile*

Présentée par :

Chahinez Hatem

Encadré par :

M. Sambeau Prak

Année : 2025-2026

Table des matières

Introduction	1
1 Présentation du projet	2
1.1 Présentation de DepanGO	2
1.2 Contexte et problématique	2
1.3 Origine du projet	2
1.4 Étude du marché et positionnement	2
1.5 Objectifs du projet	3
1.6 Réalisation du projet	3
1.7 Public cible	3
1.7.1 Les particuliers	4
1.7.2 Les techniciens	4
1.7.3 L'administrateur	4
1.8 Fonctionnement général de l'application	4
2 Analyse des besoins	5
2.1 Cahier des charges	5
2.1.1 Périmètre fonctionnel du MVP	5
2.2 Besoins fonctionnels	6
2.3 Besoins non fonctionnels	6
2.4 Contraintes	7
2.5 User stories	7
2.6 Cas d'utilisation	7
3 Gestion du projet	9
3.1 Organisation du projet	9
3.2 Méthodologie de développement	9
3.3 Utilisation de GitHub Projects	9
3.4 Gestion des versions	10
4 Conception UX/UI	11
4.1 Veille graphique	11
4.2 Charte graphique	11
4.2.1 Palette de couleurs	12
4.2.2 Le logo	12
4.2.3 Typographie	12
4.3 Maquettes	12
4.4 Intégration des maquettes	13

4.5	Responsive Design	14
4.6	Évaluation de la qualité avec Lighthouse	14
5	Architecture technique	15
5.1	Vue d'ensemble de l'architecture	15
5.2	Architecture du Frontend	15
5.3	Architecture du Backend	16
5.4	Communication et traitement d'une requête	17
6	Technologies utilisées	18
6.1	Technologies du Frontend	18
6.2	Technologies du Backend	18
6.3	Base de données	18
6.4	Authentification et sécurité	19
6.5	Envoi des e-mails	19
6.6	Outils de développement	19
7	Conception et mise en place de la base de données	20
7.1	Choix de la solution de base de données	20
7.2	Analyse des données	20
7.3	Modèle Conceptuel de Données (MCD)	20
7.4	Modèle Logique de Données (MLD)	21
7.5	Modèle Physique de Données (MPD)	21
7.6	Implémentation avec Sequelize ORM	22
7.7	Contraintes d'intégrité	22
7.8	Dictionnaire des données	22
7.8.1	Table Users	22
7.8.2	Table Services	23
7.8.3	Table Requests	23
7.8.4	Table Subscriptions	23
7.8.5	Table UserServices	24
7.8.6	Validation dans MySQL Workbench	24
8	Développement du Back-end	25
8.1	Architecture générale du Back-end	25
8.2	Fonctionnalité n°1 : Authentification sécurisée	25
8.2.1	Fonctionnement général	26
8.2.2	Route d'authentification	26
8.2.3	Vérification des identifiants	27
8.2.4	Protection des routes	27

8.3	Fonctionnalité n°2 : Création d'une demande de dépannage	28
8.3.1	Fonctionnement général	29
8.3.2	Route de création d'une demande	29
8.3.3	Traitement de la demande	29
8.3.4	Modèle de données utilisé	30
8.4	Fonctionnalité n°3 : Acceptation d'une demande et notification automatique	31
8.4.1	Fonctionnement général	31
8.4.2	Mise à jour de la demande	32
8.4.3	Notification automatique du client	32
8.4.4	Envoi de l'e-mail avec Nodemailer et Brevo	33
8.5	Synthèse de l'API REST	34
9	Développement du Front-end	35
9.1	Architecture générale du Front-end	35
9.1.1	Layouts et composants réutilisables	35
9.1.2	Navigation et routage	35
9.1.3	Communication avec l'API	36
9.2	Fonctionnalité n°1 : Authentification et accès à l'espace technicien	36
9.2.1	Connexion de l'utilisateur	36
9.2.2	Utilisation du JWT dans les requêtes Axios	37
9.2.3	Contrôle de l'accès selon le rôle	38
9.3	Fonctionnalité n°2 : Création d'une demande côté client	38
9.3.1	Gestion du formulaire	38
9.3.2	Validation des données avec Zod	39
9.4	Fonctionnalité n°3 : Consultation et acceptation d'une demande côté technicien	40
9.4.1	Récupération des demandes avec React Query	40
9.4.2	Affichage dynamique des demandes	41
9.4.3	Acceptation d'une demande	41
10	Tests et sécurité	43
10.1	Tests	43
10.1.1	Tests de l'API avec Postman	43
10.1.2	Tests fonctionnels	43
10.2	Sécurité	44
10.2.1	Hashage des mots de passe	44
10.2.2	Authentification par JWT	44
10.2.3	Contrôle des accès	44
10.2.4	Protection contre les injections SQL	45
10.2.5	Validation des données	45

10.2.6 Configuration CORS	45
10.2.7 Respect du RGPD	45
11 Déploiement	46
11.1 Installation des dépendances	46
11.2 Configuration des variables d'environnement	46
11.3 Création de la base de données	46
11.4 Lancement du Back-end	47
11.5 Lancement du Front-end	47
11.6 Vérification du fonctionnement	47
11.7 Architecture du déploiement local	47
Conclusion	49

Table des figures

1	Le Logo de l'application DepanGO	1
2	Diagramme des principaux cas d'utilisation de DepanGO	8
3	Cycle de développement inspiré de la méthode Agile	9
4	Tableau de suivi des tâches avec GitHub Projects	10
5	Dépôt GitHub du projet DepanGO	10
6	Exemples de plateforme étudiées lors de la veille graphique	11
7	Charte graphique de DepanGO	11
8	Maquettes réalisées avec Figma	13
9	Intégration de la section Hero de la page d'accueil	13
10	Résultats de l'audit Lighthouse de la page d'accueil	14
11	Vue d'ensemble de l'architecture de DepanGO	15
12	Organisation générale du Front-end de DepanGO	16
13	Organisation générale du Back-end de DepanGO	17
14	Le Modèle Conceptuel de Données (MCD) de depanGO	20
15	Le Modèle Logique de Données (MLD) de depanGO	21
16	Le Modèle Physique de Données (MPD) de depanGO	21
17	Aperçu de la base de données DepanGO dans MySQL Workbench	24
18	Architecture générale du Back-end de DepanGO	25
19	Fonctionnement de l'authentification sécurisée avec JWT	26
20	Route d'athentification (Auth.route.js)	26
21	Authentification de l'utilisateur (AuthController.js)	27
22	Vérification du jeton JWT (AuthMiddleware.js)	28
23	Cycle de création d'une demande de dépannage	29
24	Route de création d'une demande (Request.route.js)	29
25	Création d'une demande (RequestController.js)	30
26	Modèle Request (Request.js)	30
27	Cycle d'acceptation d'une demande et de notification du client	31
28	Recherche et Mise à jour d'une demande (RequestController.js)	32
29	Déclenchement de la notification automatique (RequestController.js)	32
30	Service d'envoi des e-mails (mail.js)	33
31	Organisation des routes publiques et protection de l'espace technicien (main.jsx)	35
32	Gestion de la connexion côté React (Login.jsx)	37
33	Ajout du jeton JWT aux requêtes Axios (config.js)	37
34	Protection d'une route selon le rôle (RoleGuard.jsx)	38
35	Gestion et validation du formulaire de demande (CreateRequest.jsx)	39
36	Création de la demande et gestion du résultat (CreateRequest.jsx)	40
37	Récupération et filtrage des demandes disponibles (Requests.jsx)	41

38	Acceptation d'une demande et actualisation de l'interface (Requests.jsx)	42
39	Test manuel de la route POST /auth/login avec Postman	43
40	Extrait du cahier de tests fonctionnels	44
41	Hachage des mots de passe avec bcrypt (password.js)	44
42	Configuration CORS de l'application (index.js)	45
43	Architecture locale et étapes du déploiement de l'application DepanGO	48

Liste des abréviations

Abréviation	Signification
DWWM	développeur Web et Web mobile
SMTP	Simple Mail Transfer Protocol
HTTP	HyperText Transfer Protocol
API REST	Representational State Transfer Application Programming Interface
SQL	Structured Query Language
MVC	Model View Controller
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
MPD	Modèle Physique de Données
MVP	Minimum Viable Product
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading style Sheets
HTML	HyperText Markup Language
FK	Foreign Key
PK	Primary Key
JSON	JavaScript Object Notation
JWT	JSon Web Token
ORM	Object-Relational Mapping
RGPD	Règlement Général sur la Protection des données
SEO	Search Engine Optimization
SGBDR	Système de Gestion de Base de Données Relationnelle
UI	User Interface
UX	User Experience

Introduction

Dans le cadre de la préparation du **Titre Professionnel Développeur Web et Web mobile (DWWM)**, chaque candidat est amené à concevoir et développer un projet professionnel permettant de mettre en application les compétences acquises tout au long de la formation. Ce projet constitue un élément central de l'évaluation, puisqu'il permet de démontrer la capacité à analyser un besoin, concevoir une solution technique, développer une application web complète, sécurisée et documentée.

Le présent dossier porte sur **DepanGO**, une plateforme web de mise en relation entre des particuliers ayant besoin d'un dépannage à domicile et des techniciens qualifiés. Ce projet a été imaginé afin d'apporter une réponse à une problématique fréquemment rencontrée : trouver rapidement un professionnel disponible pour intervenir lors d'une panne ou d'un incident domestique.

Au-delà de la réalisation de l'application, ce dossier présente la démarche adoptée tout au long du projet. Il décrit successivement le contexte de création, l'analyse des besoins, les choix de conception, l'architecture technique, le développement des fonctionnalités principales, la mise en place de la base de données, les mesures de sécurité ainsi que les phases de tests et les perspectives d'évolution.

L'objectif est de démontrer les compétences mobilisées pour concevoir une application web moderne répondant à un besoin réel, tout en appliquant les bonnes pratiques de développement, d'organisation et de sécurisation attendues dans un environnement professionnel.



FIGURE 1 – Le Logo de l'application DepanGO

1 Présentation du projet

1.1 Présentation de DepanGO

DepanGO est une plateforme web de mise en relation entre des particuliers ayant besoin d'un dépannage à domicile et des techniciens qualifiés. Le projet a été développé dans le cadre de la préparation du **DWWM** afin de concevoir une application web complète répondant à une problématique réelle.

La plateforme permet à un particulier de déposer rapidement une demande d'intervention sans créer de compte. Les techniciens inscrits peuvent consulter les demandes correspondantes à leurs spécialités et gérer leurs interventions depuis un espace personnel sécurisé.

L'objectif est de proposer une solution simple, rapide et intuitive facilitant la mise en relation entre les clients et les professionnels tout en centralisant la gestion des demandes de dépannage.

1.2 Contexte et problématique

Les interventions de dépannage à domicile concernent de nombreux domaines tels que la plomberie, l'électricité, le chauffage, la serrurerie ou encore la climatisation. Lorsqu'un particulier est confronté à une panne ou à une urgence, il doit souvent rechercher un professionnel disponible dans les meilleurs délais.

Les solutions existantes reposent généralement sur des recherches des annuaires en ligne, des plateformes spécialisées ou des recommandations. Ces démarches peuvent être longues et ne garantissent pas la disponibilité immédiate d'un technicien.

Les professionnels rencontrent également certaines difficultés pour développer leur activité et recevoir des demandes correspondantes à leurs compétences.

DepanGO a été imaginé afin de répondre à cette problématique en proposant une plateforme centralisant les demandes d'intervention et facilitant la mise en relation entre les particuliers et les techniciens.

1.3 Origine du projet

L'idée de DepanGO est née d'une observation issue de mon entourage.

Mon mari exerce dans le domaine de la fibre optique, des réseaux et de la domotique, et réalise régulièrement des interventions chez des particuliers. En échangeant avec lui et en observant son activité, j'ai constaté que de nombreux clients rencontrent des difficultés pour trouver rapidement un technicien disponible, notamment lorsqu'une intervention est urgente. Cette réflexion m'a conduit à imaginer DepanGO, une plateforme destinée à faciliter la mise en relation entre particuliers et professionnels du dépannage.

La réalisation de ce projet m'a également permis de mettre en pratique l'ensemble des compétences acquises au cours de la formation Développeur Web et Web Mobile, depuis l'analyse des besoins jusqu'au développement d'une application web complète.

1.4 Étude du marché et positionnement

Avant de concevoir DepanGO, une analyse des solutions existantes a été réalisée afin de mieux comprendre les pratiques du secteur de dépannage à domicile.

Parmi les plateformes étudiées notamment **MesDépanneurs.fr**, **Depanneo** et **Yoojo**. Ces services proposent la mise en relation entre particuliers et professionnels pour différents types d'interventions.

Cette étude a permis d'identifier plusieurs fonctionnalités courantes ainsi que les attentes des utilisateurs. DepanGO se positionne comme une plateforme orientée vers les interventions rapides, avec un parcours utilisateur simplifié.

Les principaux éléments différenciants sont :

- possibilité de créer une demande sans créer de compte ;
- création d'une demande en quelques étapes ;
- mise en relation avec des techniciens correspondant à la spécialité recherchée ;
- prise en compte de la disponibilité des techniciens ;
- objectif d'une intervention en moins de deux heures.

1.5 Objectifs du projet

Le principal objectif de DepanGO est de faciliter la mise en relation entre les particuliers et les techniciens afin de réduire le délai nécessaire pour obtenir une intervention.

Les principaux objectifs de DepanGO sont les suivants :

- Simplifier la recherche d'un technicien qualifié pour une intervention à domicile.
- Permettre la création rapide d'une demande de dépannage sans création de compte.
- Réduire le délai de mise en relation entre le client et le professionnel.
- Fournir aux techniciens un espace de gestion complet de leur activité.
- Développer une application responsive offrant une expérience utilisateur optimale sur ordinateur, tablette et smartphone.
- Centraliser les demandes sur une plateforme unique.
- Développer une application moderne, sécurisée et évolutive.
- Concevoir une architecture évolutive facilitant les futures améliorations de la plateforme.

1.6 Réalisation du projet

DepanGO a été développé individuellement dans le cadre de la formation DWWM.

Le choix de réaliser ce projet seule avait pour objectif de mettre en pratique l'ensemble des compétences acquises au cours de la formation. J'ai ainsi participé à toutes les étapes du projet : analyse des besoins, conception, modélisation de la base de données, développement du front-end et du back-end, mise en place de la sécurité, tests et documentation.

Cette approche m'a permis d'acquérir une vision globale du développement d'une application web complète et de renforcer mon autonomie dans la conduite d'un projet.

1.7 Public cible

DepanGO s'adresse principalement à trois catégories d'utilisateurs, chacune disposant de besoins spécifiques et d'un niveau d'accès différent au sein de l'application.

1.7.1 Les particuliers

Les particuliers utilisent DepanGO lorsqu'ils rencontrent un problème nécessitant l'intervention rapide d'un professionnel à domicile pour des services tels que :

- plomberie
- électricité
- serrurerie
- chauffage
- Internet

Afin de simplifier leur expérience, aucune création de compte n'est imposée.

1.7.2 Les techniciens

Les techniciens doivent créer un compte afin d'accéder aux différentes fonctionnalités qui leur sont réservées.

Après authentification, ils disposent d'un espace personnel leur permettant de :

- consulter les demandes correspondantes à leurs spécialités ;
- accepter ou refuser une intervention ;
- gérer leurs interventions en cours ;
- consulter leur historique ;
- modifier leurs informations personnelles ;
- gérer leurs spécialités ;
- modifier leur disponibilité ;
- consulter l'état de leur abonnement.

1.7.3 L'administrateur

L'administrateur est chargé de superviser le fonctionnement de l'application et de gérer ses principales ressources, notamment :

- les utilisateurs ;
- les techniciens ;
- les services proposés ;
- les demandes d'intervention ;
- les abonnements.

1.8 Fonctionnement général de l'application

Le fonctionnement de DepanGO repose sur un processus simple.

Le particulier complète un formulaire de demande en renseignant les informations relatives à son intervention. Une fois enregistrée, la demande devient accessible aux techniciens concernés.

Les techniciens consultent les demandes disponibles, acceptent une intervention lorsqu'ils sont disponibles puis assurent son suivi depuis leur tableau de bord.

L'ensemble du processus permet de simplifier la gestion des demandes tout en réduisant le délai de mise en relation entre les particuliers et les professionnels.

2 Analyse des besoins

2.1 Cahier des charges

Dès le début du projet, il était clair que toutes les fonctionnalités envisagées ne pourraient pas être développées dans le temps imparti de la formation DWWM.

L'objectif était donc de concevoir un **MVP (Minimum Viable Product)** intégrant les fonctionnalités essentielles, tout en prévoyant des évolutions futures.

Le cahier des charges a servi de référence tout au long du développement et a évolué en fonction des contraintes rencontrées. Certaines fonctionnalités ont été reportées afin de se concentrer sur les besoins prioritaires des utilisateurs.

Cette première version de DepanGO permet aux particuliers de créer une demande de dépannage et aux techniciens de gérer leur activité. Les versions futures intégreront notamment **le paiement des interventions par les particuliers et le paiement des abonnements par les techniciens**, ainsi que d'autres fonctionnalités destinées à faire évoluer l'application.

2.1.1 Périmètre fonctionnel du MVP

Le développement de DepanGO s'est concentré sur les fonctionnalités essentielles du MVP. Les autres fonctionnalités seront intégrées dans les futures versions de l'application.

Fonctionnalités prévues	Réalisées dans le MVP (V1)	Prévues pour une version ultérieure
Création d'une demande de dépannage sans création de compte	✓	
Inscription et connexion des techniciens	✓	
Gestion du profil technicien	✓	
Gestion des spécialités	✓	
Gestion de la disponibilité	✓	
Consultation des demandes selon les spécialités	✓	
Acceptation d'une intervention	✓	
Gestion des abonnements	✓	
Envoi automatique des coordonnées du technicien par e-mail	✓	
Tableau de bord administrateur	✓	
Gestion des utilisateurs		✓
Gestion des services		✓
Gestion des demandes		✓
Paiement en ligne des abonnements par les techniciens		✓
Paiement des interventions par les particuliers		✓
Système d'avis et de notation		✓
Notifications en temps réel		✓
Géolocalisation des techniciens		✓

TABLE 2 – Périmètre fonctionnel du MVP de DepanGO

2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités que DepanGO doit offrir afin de répondre aux besoins des différents utilisateurs de l'application.

Utilisateur	Besoins fonctionnels
Particulier	• consulter les services proposés ; • comprendre le fonctionnement de la plateforme ; • créer une demande de dépannage sans créer de compte ; • transmettre les informations nécessaires au traitement de sa demande. • Renseigner les informations nécessaires à la demande d'intervention ; • Recevoir les coordonnées du technicien après l'acceptation de la demande.
Technicien	• créer un compte ; • se connecter à son espace personnel ; • consulter les demandes correspondant à ses spécialités ; • accepter une intervention ; • gérer ses interventions ; • consulter les interventions terminées ; • modifier son profil ; • gérer ses spécialités ; • modifier sa disponibilité ; • consulter son abonnement.
Administrateur	• Gérer les abonnements des techniciens

TABLE 3 – Besoins fonctionnels de l'application

2.3 Besoins non fonctionnels

En complément des fonctionnalités attendues, plusieurs exigences de qualité ont été prises en compte lors de la conception de DepanGO.

L'application doit être :

- simple d'utilisation ;
- responsive sur ordinateur, tablette et smartphone ;
- sécurisée ;
- performante ;
- facilement maintenable,
- évolutive afin de permettre l'ajout de nouvelles fonctionnalités.

2.4 Contraintes

Type de contrainte	Description
Temporelle	Le projet devait être réalisé dans le délai imparti par la formation DWWM.
Technique	Séparation entre le frontend, le backend et la base de données.
Sécurité	Protéger les données et sécuriser l'accès des utilisateurs authentifiés.
Responsive	Compatibilité avec les principaux appareils.
Performance	Temps de réponse rapide pour les principales fonctionnalités.
Évolutivité	Architecture permettant l'ajout de nouvelles fonctionnalités.

TABLE 4 – Contraintes

2.5 User stories

Utilisateur	User story
Particulier	En tant que particulier, je souhaite : • déposer une demande afin de trouver rapidement un technicien disponible. • consulter les services afin de choisir le dépannage correspondant à mon besoin.
Technicien	En tant que technicien, je souhaite : • consulter uniquement les demandes correspondant à mes spécialités. • accepter une intervention afin de prendre en charge un client. • gérer mon profil et mes spécialités.
Administrateur	En tant qu'administrateur, je souhaite : • gérer les utilisateurs, les services, les demandes et les abonnements.

TABLE 5 – User stories

2.6 Cas d'utilisation

Les cas d'utilisation permettent de représenter les interactions entre les différents utilisateurs et l'application.

Le diagramme ci-dessous présente les principaux cas d'utilisation de DepanGO pour les différents acteurs de l'application :

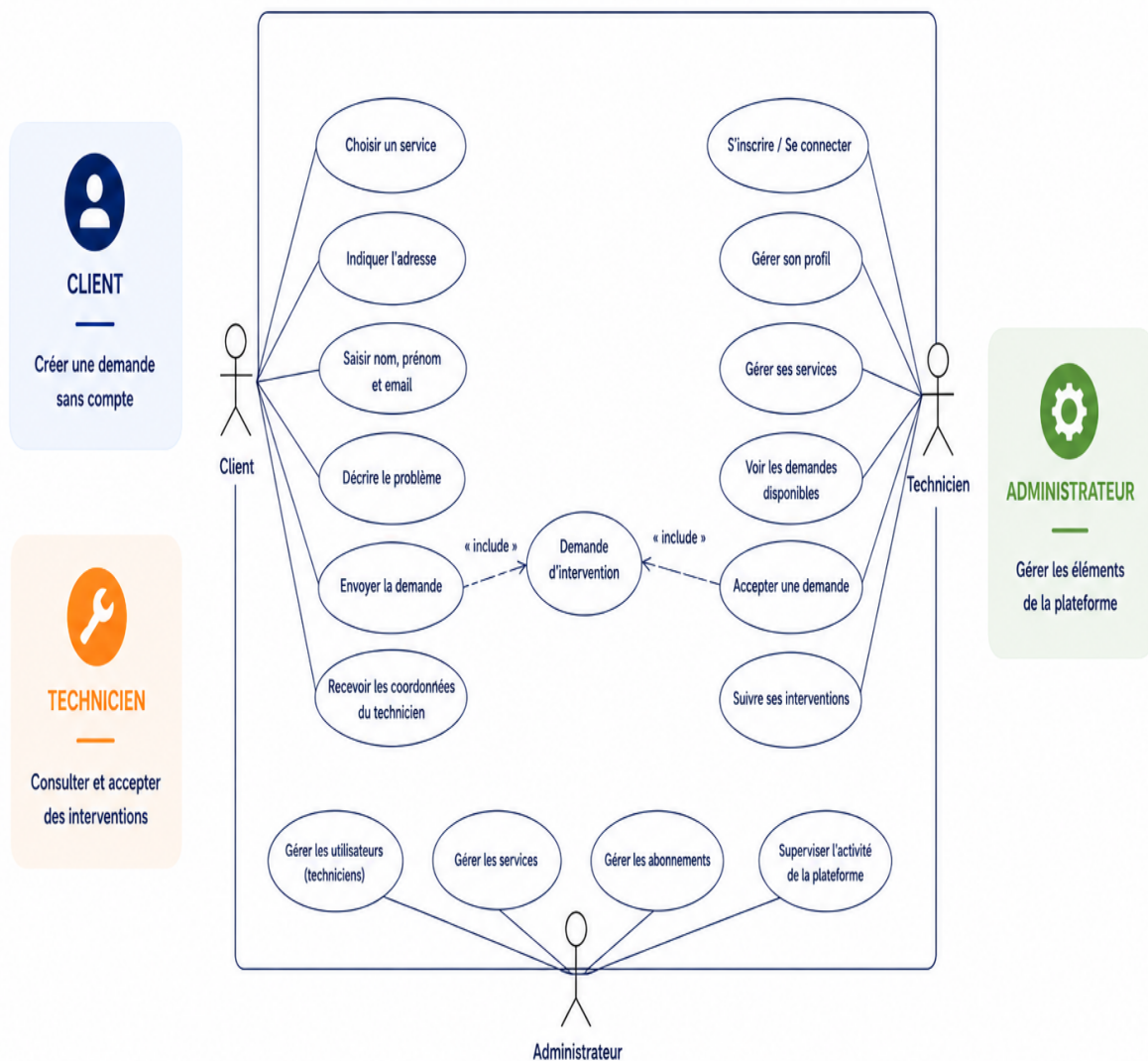


FIGURE 2 – Diagramme des principaux cas d'utilisation de DepanGO

3 Gestion du projet

3.1 Organisation du projet

Afin de mener le développement de DepanGO de manière structurée, le projet a été organisé en plusieurs phases successives. Cette organisation a permis de valider chaque étape avant de passer à la suivante et d'assurer un suivi régulier de l'avancement du projet.

Le déroulement du projet a suivi les principales étapes suivantes :

1. Analyse des besoins.
2. Réalisation des maquettes de l'application.
3. Conception de la base de données.
4. Développement du backend et de L'API REST
5. Tests des endpoints avec Postman
6. Développement du frontend React.
7. tests fonctionnels de l'application.
8. Adaptation du responsive design.
9. Corrections, optimisations et finalisation du projet.

3.2 Méthodologie de développement

Le développement de DepanGO s'est appuyé sur une démarche inspirée des méthodes Agiles, adaptée à un projet réalisé individuellement.

Les fonctionnalités ont été développées progressivement, par itérations successives. Chaque nouvelle fonctionnalité était conçue, développée, testée puis validée avant le démarrage de la suivante. Cette approche a permis d'obtenir une application évolutive tout en limitant les risques liés aux modifications importantes en fin de projet.

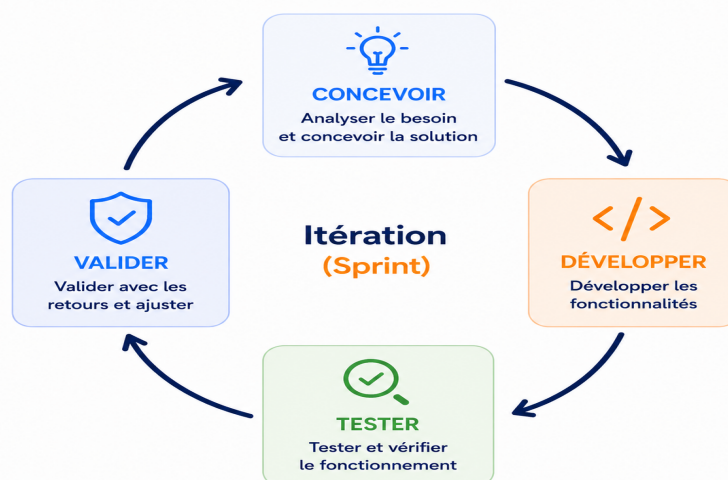


FIGURE 3 – Cycle de développement inspiré de la méthode Agile

3.3 Utilisation de GitHub Projects

Pour organiser le développement, GitHub Projects a été utilisé comme outil de gestion de projet.

Les différentes fonctionnalités ont été découpées en tâches puis réparties dans plusieurs sprints. Chaque tâche suivait un cycle d'avancement représenté par différentes colonnes :

- **Backlog** : tâches planifiées mais non commencées ;
- **Ready** : tâches prêtes à être développées ;
- **In Progress** : tâches en cours de développement ;
- **In Review** : fonctionnalités en cours de vérification ou de validation ;
- **Done** : fonctionnalités terminées et validées.

Cette organisation a permis de visualiser rapidement l'état d'avancement du projet, de prioriser les développements et de conserver une vue d'ensemble sur les fonctionnalités restantes.

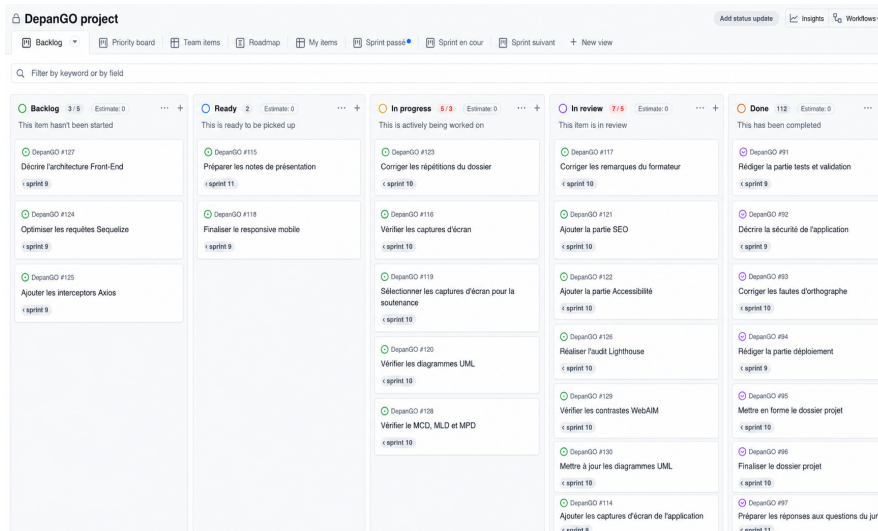


FIGURE 4 – Tableau de suivi des tâches avec GitHub Projects

3.4 Gestion des versions

Le projet a été versionné à l'aide de Git et hébergé sur GitHub.

Chaque évolution importante du projet a été enregistrée sous forme de commits, permettant de conserver un historique complet des modifications réalisées. Cette approche facilite le suivi des évolutions, la correction d'éventuelles erreurs et le retour à une version précédente si nécessaire.

L'utilisation de GitHub a également permis de centraliser le code source du projet et d'assurer une sauvegarde continue de son évolution.

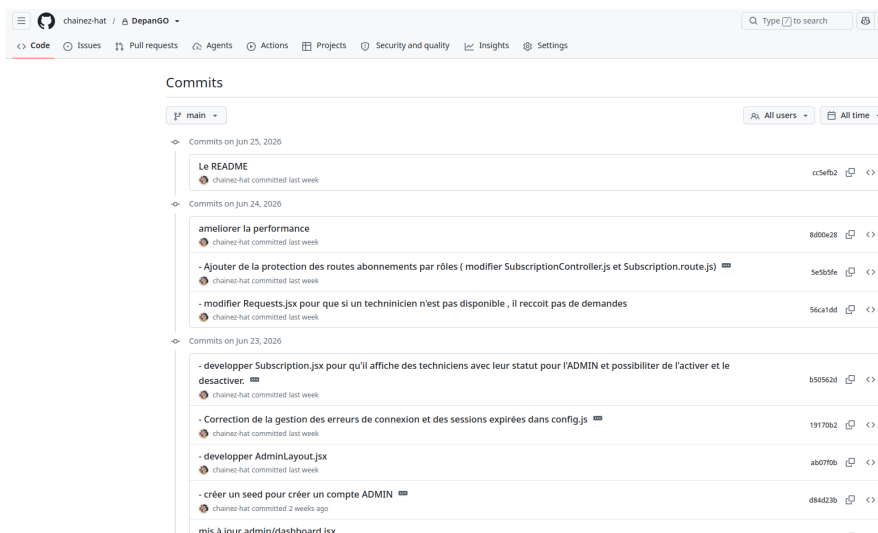


FIGURE 5 – Dépôt GitHub du projet DepanGO

4 Conception UX/UI

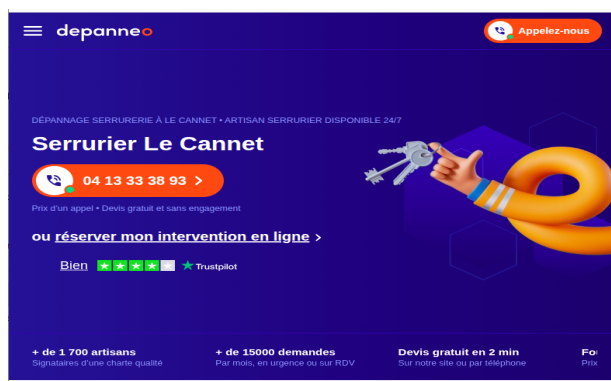
4.1 Veille graphique

Avant de concevoir l'identité visuelle de DepanGO, une veille graphique a été réalisée afin d'analyser les codes visuels utilisés par des plateformes du même secteur, notamment Depanneo et MesDépanneurs.fr.

Cette analyse a porté sur plusieurs éléments : les couleurs, les boutons, les bordures, les cartes, les icônes ainsi que la mise en valeur des actions importantes.

Elle a permis de constater que le bleu est largement utilisé pour transmettre une image de confiance et de professionnalisme, tandis que des couleurs plus vives, comme l'orange, servent à attirer l'attention sur les actions principales.

Sans reproduire ces interfaces, cette veille a servi de source d'inspiration pour définir la charte graphique de DepanGO.



(a) La plateforme Depanneo



(b) La plateforme MesDépanneurs.fr

FIGURE 6 – Exemples de plateforme étudiées lors de la veille graphique

4.2 Charte graphique

A la suite de la veille graphique, une charte graphique a été définie afin de créer une identité visuelle cohérente pour DepanGO.

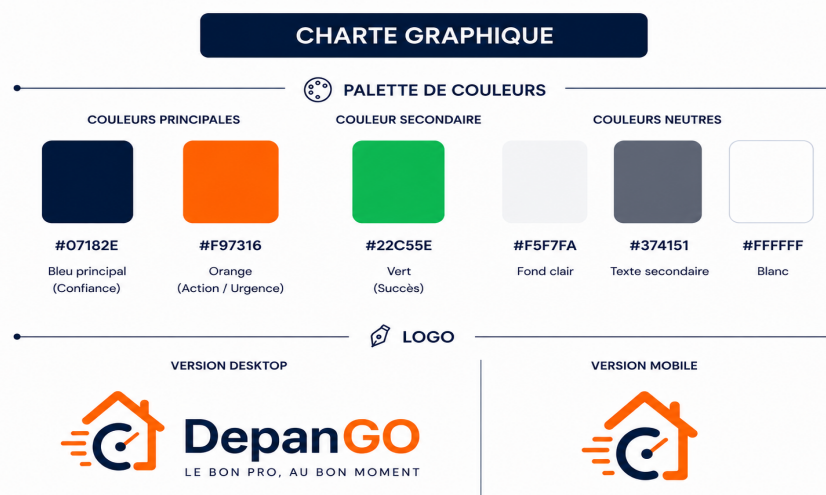


FIGURE 7 – Charte graphique de DepanGO

4.2.1 Palette de couleurs

La palette de couleurs a été choisie afin de refléter les valeurs de l'application :

Couleur	Utilisation	Signification
Bleu nuit #07182E	En-têtes, navigation, titres	Confiance, professionnalisme
Orange #F97316	Boutons, appels à l'action, icônes	Rapidité, action, urgence
Blanc #FFFFFF	Arrière-plan principal	Lisibilité et simplicité
Gris clair #F5F7FA	Fonds secondaires	Hierarchie visuelle
Vert #22C55E	Messages de succès	Validation et confirmation

TABLE 6 – Palette de couleurs

4.2.2 Le logo

Le logo de DepanGO reflète l'identité de l'application. Il associe plusieurs éléments graphiques :

- **une maison** représentant le domicile ;
- **un compteur et des lignes de mouvement** symbolisant la rapidité d'intervention ;
- **une palette de couleurs** évoquant la confiance et la réactivité.

Afin de s'adapter aux différents supports, le logo est disponible en deux versions :

- **Version desktop** : pictogramme, nom DepanGO et slogan.
- **Version mobile** : pictogramme uniquement.

4.2.3 Typographie

La police **Poppins** a été choisie pour son **style moderne**, sa **lisibilité** et sa **cohérence visuelle** sur l'ensemble de l'application.

4.3 Maquettes

Avant le développement, les interfaces de DepanGO ont été conçues avec **Figma** afin de définir l'organisation des pages et d'assurer une expérience utilisateur cohérente.

Les maquettes ont permis de :

- concevoir les principaux écrans de l'application ;
- créer des composants réutilisables (boutons, cartes, icônes et logo) ;
- harmoniser l'identité visuelle de l'interface ;
- adapter les interfaces aux formats **desktop** et **mobile**.

Cette étape a permis de valider la conception des interfaces avant leur intégration dans l'application.

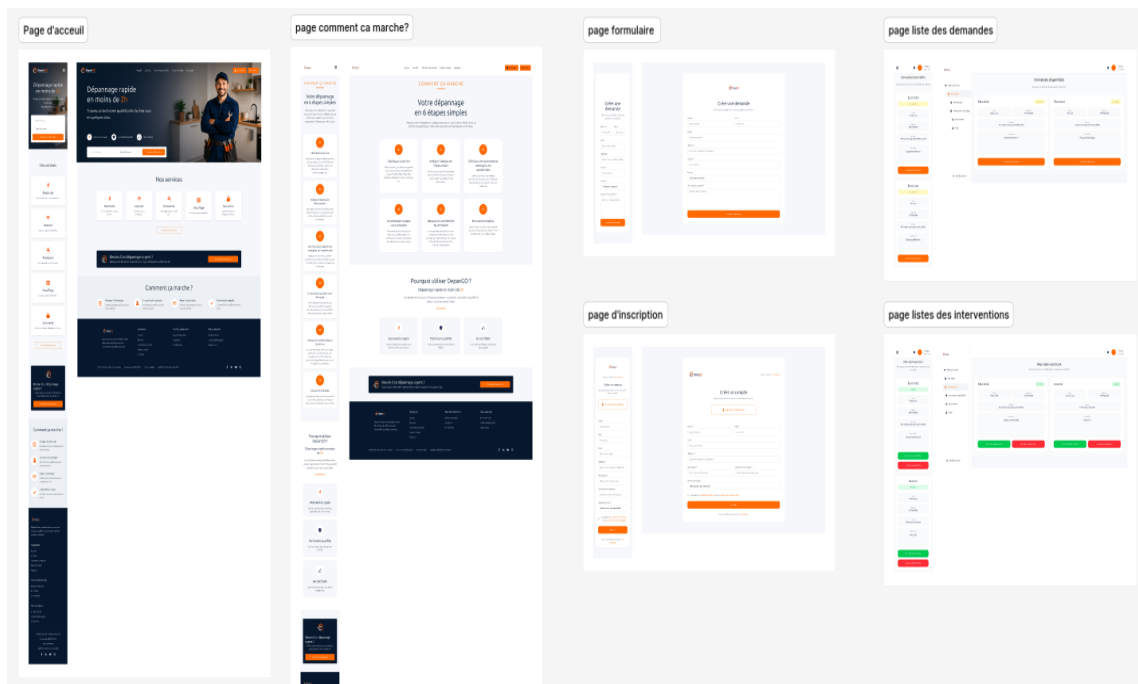


FIGURE 8 – Maquettes réalisées avec Figma

4.4 Intégration des maquettes

Une fois les maquettes validées, elles ont été intégrées dans l'application. L'objectif était de reproduire les interfaces conçues sous Figma tout en garantissant une application responsive et facilement maintenable.

L'intégration a consisté à :

- structurer les pages avec des **balises HTML** adaptées ;
- appliquer les styles avec **Tailwind CSS** en respectant la charte graphique ;
- créer des composants réutilisables afin d'assurer une cohérence entre les différentes pages ;
- adapter les interfaces aux différentes tailles d'écran grâce aux classes responsives de Tailwind CSS.

La figure suivante présente un exemple d'intégration de la section Hero de la page d'accueil, qui est une page statique de l'application. Elle met en parallèle un extrait du code d'intégration et le résultat obtenu dans l'application.

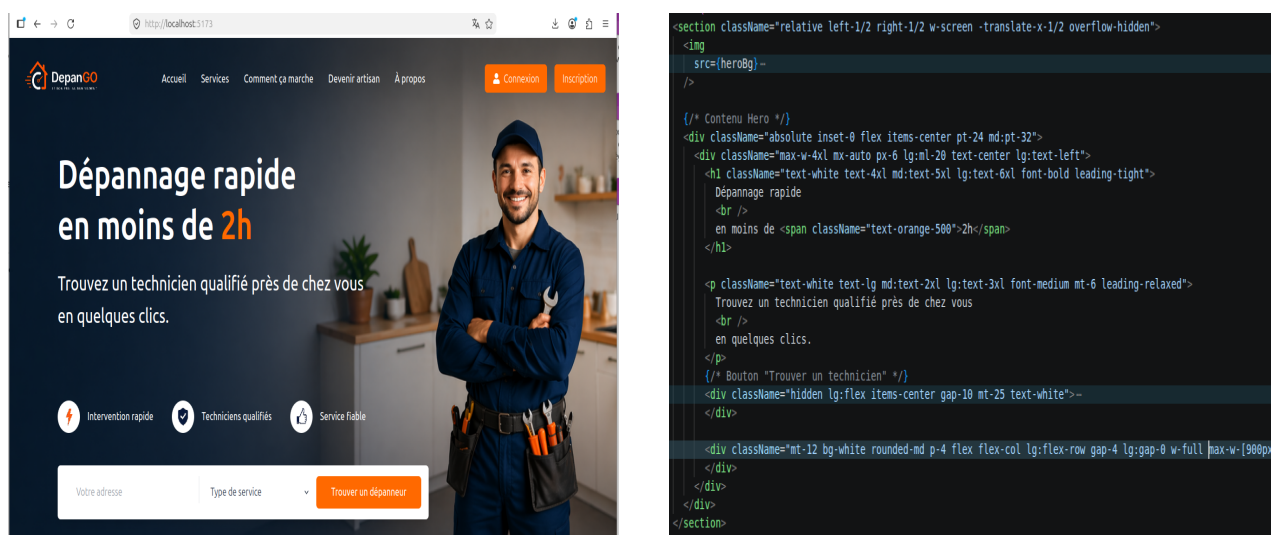


FIGURE 9 – Intégration de la section Hero de la page d'accueil

4.5 Responsive Design

Le responsive design a été mis en oeuvre avec les classes responsives de **Tailwind CSS** afin d'adapter automatiquement l'interface aux différentes tailles d'écran.

Pour cela, les **breakpoints** de Tailwind CSS ont été utilisés, notamment **md** (à partir de 768 px) et **lg** (à partir de 1024 px), permettant de modifier la mise en page et l'affichage des éléments en fonction de la largeur de l'écran.

Les principaux ajustements réalisés sont :

- adaptation de la taille des textes selon les breakpoints **md** et **lg**;
- passage d'une disposition verticale à horizontale avec **flex-col** et **lg : flex-row**;
- affichage ou masquage de certains éléments grâce aux classes **hidden** et **lg : flex**;
- conservation d'une navigation fluide et d'une bonne lisibilité sur les versions desktop, tablette et mobile.

4.6 Évaluation de la qualité avec Lighthouse

Afin d'évaluer la qualité de l'application web, un audit a été réalisé à l'aide de l'outil **Lighthouse**, intégré aux outils de développement de Google Chrome.

- **Performance** : chargement des données à la demande grâce à React Query ;
- **Accessibilité** : utilisation d'attributs **alt** descriptifs sur les images ;
- **Bonnes pratiques** : respect des bonnes pratiques de développement ;
- **SEO** : utilisation d'une structure HTML sémantique avec un **h1** unique par page et des balises **nav**, **main** et **section**.

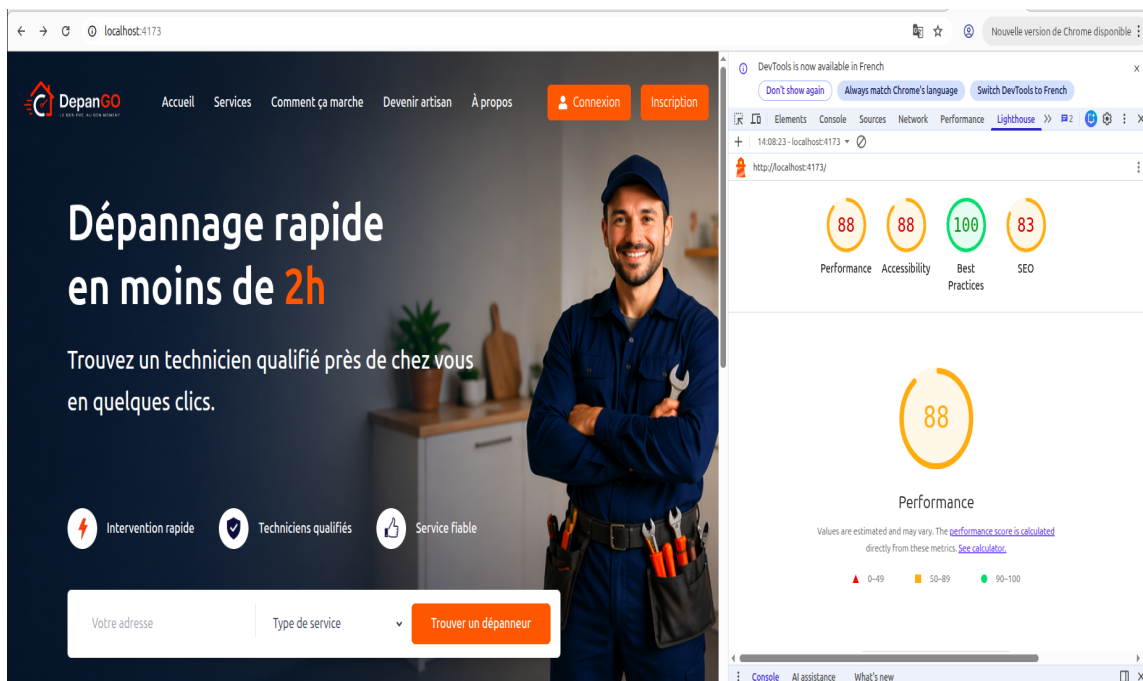


FIGURE 10 – Résultats de l'audit Lighthouse de la page d'accueil

5 Architecture technique

5.1 Vue d'ensemble de l'architecture

DepanGO repose sur une architecture client-serveur composée de quatre éléments principaux :

- Frontend : interface utilisateur développée avec React ;
- Backend : API REST développée avec Node.js et Express ;
- Base de données relationnelle MySQL : stockage des données ;
- service SMTP Brevo : envoi automatique des e-mails.

Le frontend gère l'affichage des interfaces et les interactions utilisateur. Les requêtes sont transmises à l'API REST qui, pour les routes protégées, s'appuie sur un middleware d'authentification afin de vérifier le jeton JWT et les autorisations d'accès. Le backend applique ensuite la logique métier, interagit avec la base de données puis retourne une réponse au format JSON. Certaines opérations d'une demande d'intervention, déclenchent également l'envoi automatique d'un e-mail via Brevo SMTP.

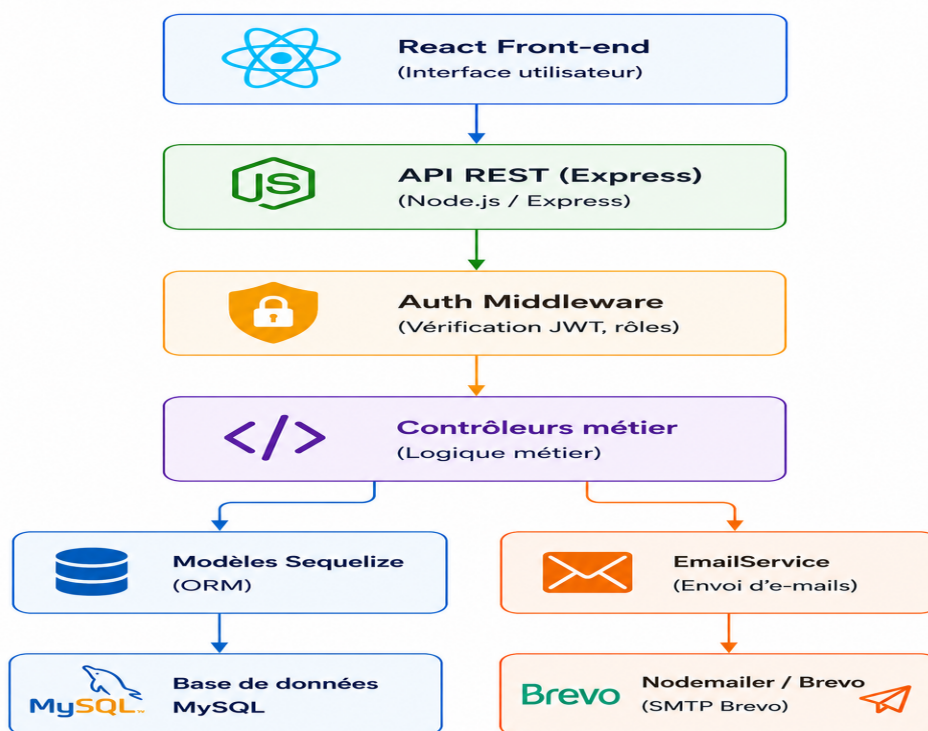


FIGURE 11 – Vue d'ensemble de l'architecture de DepanGO

5.2 Architecture du Frontend

Le front-end de DepanGO est organisé en plusieurs dossiers afin de séparer les responsabilités et de faciliter la maintenance de l'application. Cette organisation favorise la modularité du code et la réutilisation des composants.

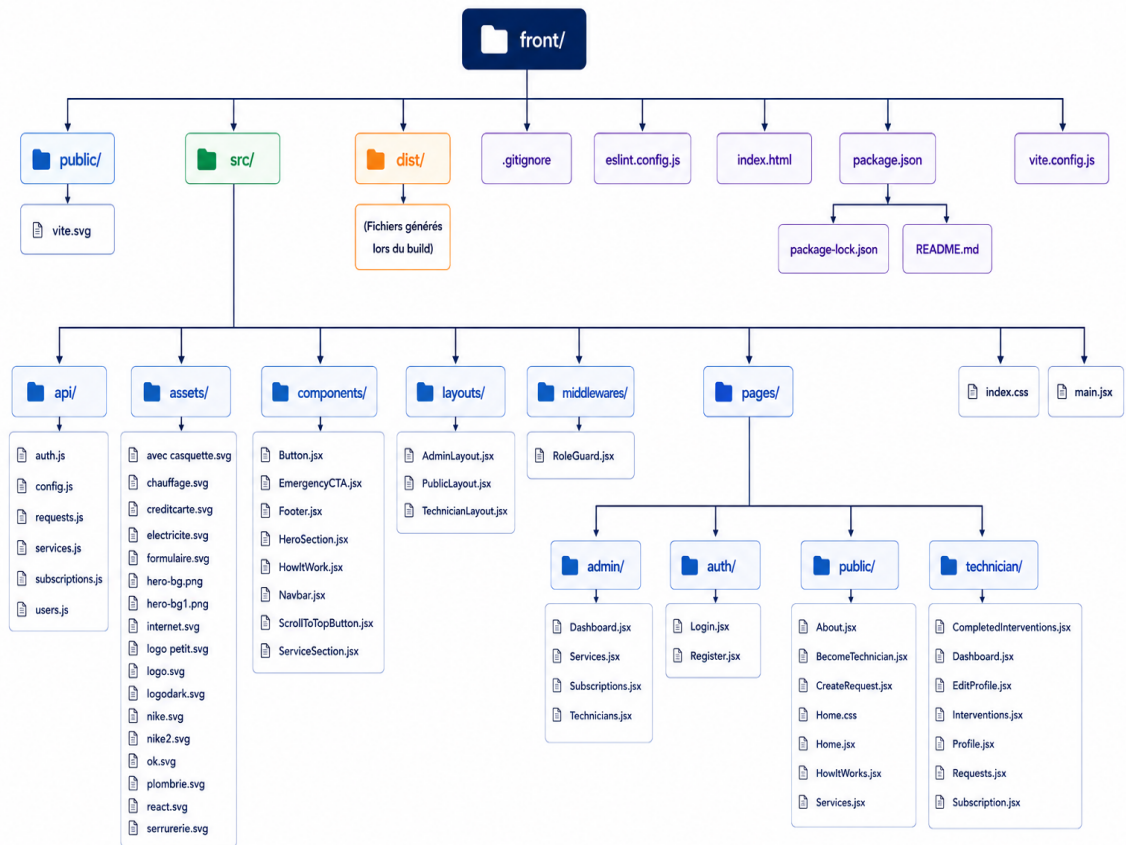


FIGURE 12 – Organisation générale du Front-end de DepanGO

Les principaux dossiers sont les suivants :

Dossier	Rôle
api	Communication avec l'API REST
assets	Images, logos, icônes
components	Composants réutilisables
layouts	Organisation des différentes interfaces
middlewares	Protection des routes (RoleGuard)
pages	Ensemble des pages de l'application

TABLE 7 – Les principaux dossiers de l'architecture du Frontend

5.3 Architecture du Backend

Le back-end de DepanGO est organisé selon une architecture inspirée du modèle **MVC (Model-View-Controller)**. Les différents dossiers permettent de séparer les responsabilités entre la gestion des requêtes, la logique métier, l'accès aux données et les services techniques.

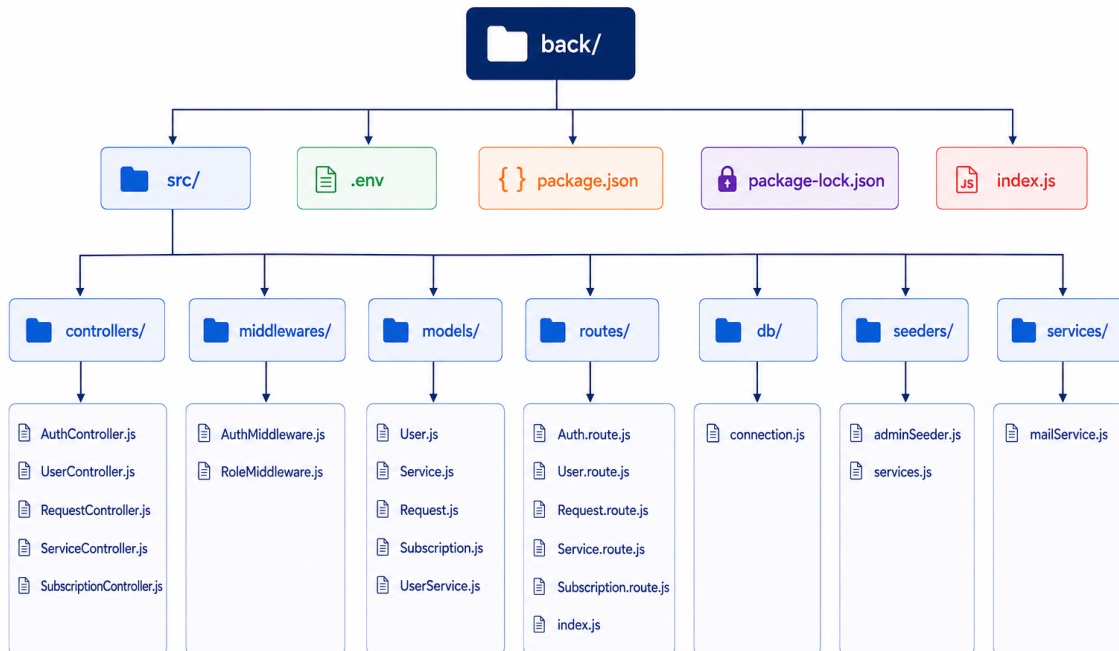


FIGURE 13 – Organisation générale du Back-end de DepanGO

Les principaux dossiers sont les suivants :

Dossier	Rôle
controllers	Contiennent la logique métier
models	Définissent les modèles Sequelize
routes	Définissent les endpoints de l'API
middlewares	Authentification, autorisation et sécurité
db	Configuration de la connexion MySQL
utils	Fonctions utilitaires (envoi d'e-mails, gestion des mots de passe)
seeds	Données initiales de la base

TABLE 8 – Les principaux dossiers de l'architecture du Backend

Cette séparation permet de limiter les dépendances entre les différentes couches de l'application et facilite les évolutions futures.

5.4 Communication et traitement d'une requête

Les échanges entre le frontend et le backend sont réalisés à travers une **API REST**. Les requêtes HTTP sont envoyées à l'aide d'**Axios**, tandis que **React Query** assure la gestion des requêtes asynchrones et la synchronisation des données.

Lorsqu'un utilisateur réalise une action, le traitement suit les étapes suivantes :

1. Le frontend valide les données du formulaire à l'aide de React Hook Form et de Zod.
2. Axios envoie une requête HTTP vers l'API REST.
3. La route Express reçoit la requête et la dirige vers le contrôleur concerné.
4. Le contrôleur applique la logique métier et effectue les validations nécessaires.
5. Sequelize interagit avec la base de données MySQL.
6. Une réponse au format JSON est renvoyée au frontend.
7. React Query met automatiquement l'interface à jour.
8. Si nécessaire, un e-mail est envoyé via Nodemailer et Brevo SMTP.

6 Technologies utilisées

6.1 Technologies du Frontend

Le tableau suivant présente les différentes technologies retenues pour le développement du frontend ainsi que leur rôle au sein de l'application :

Technologie	Utilisation dans DepanGO
React	Développement de l'interface utilisateur
Vite	Initialisation et compilation du projet
React Router	Navigation entre les pages
Axios	Communication avec l'API REST
React Query	Gestion des requêtes et du cache
React Hook Form	Gestion des formulaires
Zod	Validation des données
Tailwind CSS	Mise en page et Responsive Design
React Hot Toast	Affichage des notifications

TABLE 9 – Technologies utilisées pour le développement du frontend

6.2 Technologies du Backend

Les principales technologies utilisées pour le développement du backend sont présentées dans le tableau suivant :

Technologie	Utilisation dans DepanGO
Node.js	Environnement d'exécution JavaScript
Express.js	Création de l'API REST
Sequelize	ORM pour MySQL
JWT	Authentification des utilisateurs
bcrypt	Hachage des mots de passe
Nodemailer	Envoi des e-mails
Brevo SMTP	Serveur SMTP

TABLE 10 – Technologies utilisées pour le développement du backend

6.3 Base de données

Technologie	Utilisation
MySQL	Stockage des données
Sequelize	Gestion des modèles et relations

TABLE 11 – Technologies utilisées pour la base de données

6.4 Authentification et sécurité

Technologie	Rôle
JWT	Génération du token de connexion
bcrypt	Hachage des mots de passe
RoleGuard	Protection des pages React
AuthMiddleware	Protection des routes Express

TABLE 12 – Technologies utilisées pour l’authentification et la sécurité

6.5 Envoi des e-mails

Technologie	Rôle
Nodemailer	Création et envoi des e-mails
Brevo SMTP	Distribution des e-mails

TABLE 13 – Technologies utilisées pour l’envoi des e-mails

6.6 Outils de développement

Outil	Utilisation
Visual Studio Code	Développement
Git	Versionnement
GitHub	Hébergement du code
GitHub Projects	Gestion des sprints
Postman	Tests de l’API
Figma	Maquettage

TABLE 14 – outils utilisés pour le développement de l’application

7 Conception et mise en place de la base de données

7.1 Choix de la solution de base de données

La base de données constitue le coeur de l'application DepanGO. Sa conception a été réalisée à l'aide de la méthode **Merise**, qui permet de structurer les données avant leur implémentation. Cette modélisation s'appuie sur trois niveaux : le **Modèle Conceptuel de Données (MCD)**, le **Modèle Logique de Données (MLD)** et le **Modèle Physique de Données (MPD)**.

Pour l'implémentation, le choix s'est porté sur **MySQL**, un système de gestion de bases données relationnelles reconnu pour sa fiabilité, ses performances et sa compatibilité avec les applications web.

Afin de faciliter le développement, **Sequelize ORM** a été utilisé pour représenter les tables sous forme de modèles JavaScript, gérer les relations entre les entités et simplifier les opérations de manipulation des données.

7.2 Analyse des données

Après l'étude des besoins fonctionnels, quatre entités principales ont été identifiées :

Entité	Description
Utilisateurs	Représente les comptes des administrateurs et des techniciens
Services	Représente les différents types de dépannage proposés sur la plateforme.
Demandes	Représente les demandes d'intervention créées par les clients.
Abonnements	Représente les abonnements souscrits par les techniciens.

TABLE 15 – Les entités principales

7.3 Modèle Conceptuel de Données (MCD)

Le modèle Conceptuel de Données (MCD) permet de représenter les principales entités de l'application ainsi que les relations qui les relient, indépendamment de toute contrainte technique. Il constitue la base de la conception de la base de données.

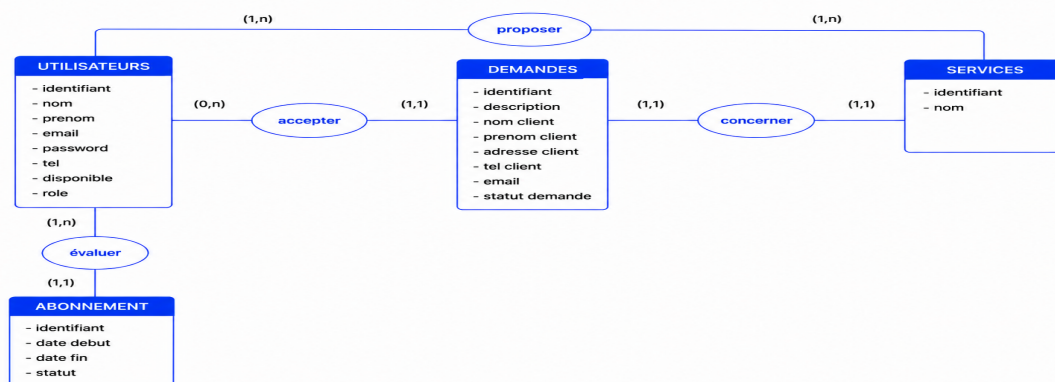


FIGURE 14 – Le Modèle Conceptuel de Données (MCD) de depanGO

7.4 Modèle Logique de Données (MLD)

Le modèle Logique de Données (MLD) traduit le MCD en un modèle relationnel. Les entités deviennent des tables, les relations sont représentées par des clés primaires et des clés étrangères, et les relations plusieurs-à-plusieurs (N,N) sont transformées en tables pivot.

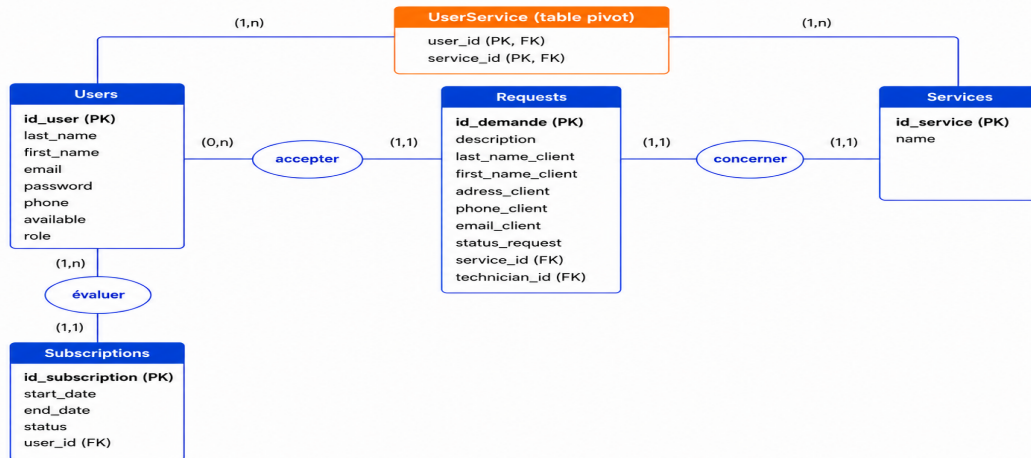


FIGURE 15 – Le Modèle Logique de Données (MLD) de depanGO

7.5 Modèle Physique de Données (MPD)

Le Modèle Physique de Données (MPD) correspond à l'implémentation concrète de la base de données dans le **SGBDR MySQL**. Il définit les types de données de chaque colonne, les clés primaires, les clés étrangères ainsi que les contraintes nécessaires au bon fonctionnement de l'application.

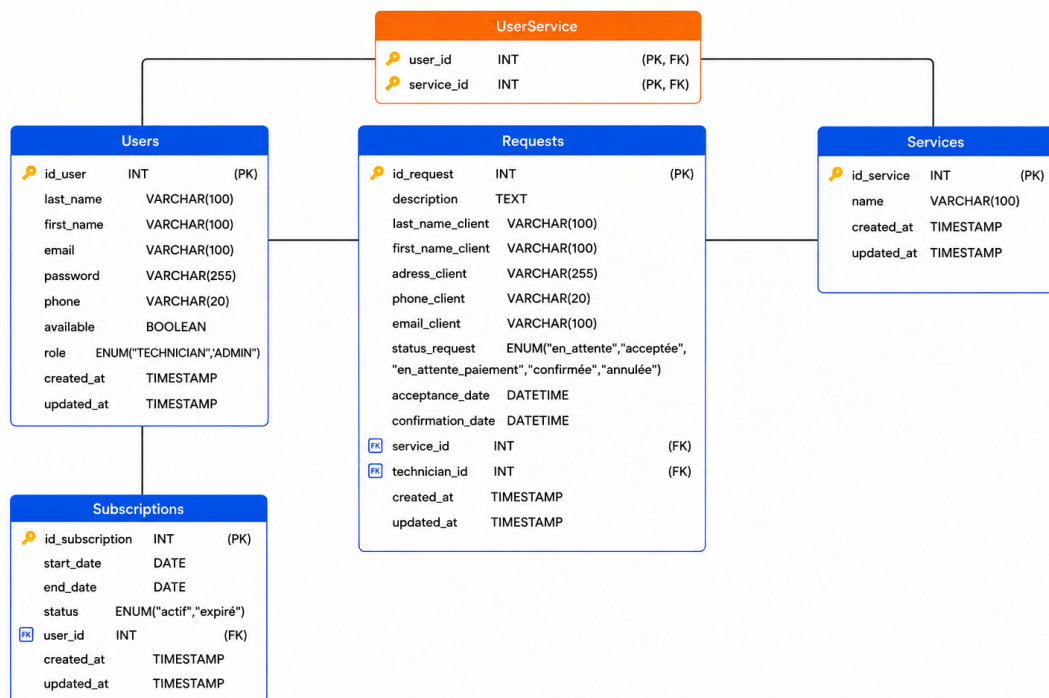


FIGURE 16 – Le Modèle Physique de Données (MPD) de depanGO

7.6 Implémentation avec Sequelize ORM

L'implémentation de la base de données a été réalisée à l'aide de **Sequelize ORM**. Chaque table du modèle physique est représentée par un modèle JavaScript, auquel sont associées les relations nécessaires au fonctionnement de l'application. Les principales associations utilisées sont les suivantes :

Relation	Méthode Sequelize	Description
Users $\iff$ Services	<code>belongsToMany()</code>	Relation plusieurs-à-plusieurs via la table pivot <code>UserServices</code> .
Services $\iff$ Requests	<code>hasMany()</code> / <code>belongsToMany()</code>	Un service peut être associé à plusieurs demandes.
Users $\iff$ Requests	<code>hasMany()</code> / <code>belongsToMany()</code>	Un technicien peut accepter plusieurs demandes.
Users $\iff$ Subscriptions	<code>belongsTo()</code>	Association entre un utilisateur et son abonnement.

TABLE 16 – Les différentes associations utilisées

Cette implémentation facilite la manipulation des données, garantit la cohérence des relations entre les modèles et simplifie les opérations de création, de lecture, de modification et de suppression (CRUD)

7.7 Contraintes d'intégrité

Afin de garantir la cohérence et l'intégrité des données, plusieurs contraintes ont été mises en place dans la base de données.

Contrainte	Utilisation
Primary Key (PK)	Identification unique de chaque enregistrement.
Foreign Key (FK)	Maintien des relations entre les tables.
UNIQUE	Empêche les doublons sur certains champs (ex : email).
NOT NULL	Rend certains champs obligatoires.
DEFAULT	Définit une valeur par défaut.
ENUM	Limite les valeurs possibles pour certains champs (rôle, statut, etc.).

TABLE 17 – Les différentes contraintes d'intégrité

7.8 Dictionnaire des données

7.8.1 Table Users

Champ	Type	Description
id	INT	Identifiant de l'utilisateur
last_name	VARCHAR	Nom
first_name	VARCHAR	Prénom
email	VARCHAR	Adresse électronique
phone	VARCHAR	Téléphone
password	VARCHAR	Mot de passe chiffré
role	ENUM	ADMIN ou TECHNICIEN
available	BOOLEAN	Disponibilité du technicien

TABLE 18 – Table Utilisateurs

7.8.2 Table Services

Champ	Type	Description
id	INT	Identifiant du service
name	VARCHAR	Nom du service

TABLE 19 – Table Services

7.8.3 Table Requests

Champ	Type	Description
id	INT	Identifiant de la demande
description	TEXT	Description du problème
last_name_client	VARCHAR	Nom du client
first_name_client	VARCHAR	Prénom du client
adresse_client	VARCHAR	Adresse
phone_client	VARCHAR	Téléphone
email_client	VARCHAR	Adresse e-mail
status_request	ENUM	Etat de la demande
service_id	INT	Service demandé
technicien_id	INT	Technicien demandé

TABLE 20 – Table Demandes

7.8.4 Table Subscriptions

Champ	Type	Description
id	INT	Identifiant
start_date	DATE	Date de début
end_date	DATE	Date de fin

status	ENUM	Status de l'abonnement
user_id	INT	Technicien concerné

TABLE 21 – Table Abonnements

7.8.5 Table UserServices

Champ	Type	Description
user_id	INT	Référence vers un technicien
service_id	INT	Référence vers un service

TABLE 22 – Table Services Utilisateur

7.8.6 Validation dans MySQL Workbench

Une fois les modèles Sequelize créés, les tables ont été générées dans MySQL. La structure de la base de données a ensuite été vérifiée à l'aide de MySQL Workbench, permettant de contrôler les relations entre les tables et la conformité du modèle physique.

The screenshot shows the MySQL Workbench interface. The 'Schemas' sidebar on the left lists the database structure: DepanGO (Tables: Requests, Services, Subscriptions, Users), UserServices, Views, Stored Procedures, Functions, marsai, and sys. The main editor shows a SQL query: `SELECT * FROM DepanGO.Users;`. The 'Result Grid' at the bottom displays the following data:

#	id	last_name	first_name	email	phone	password	role	available	createdAt	updatedAt
1	1	Dupont	Jean	jean@test.com	0612345678	\$2b\$10\$gUKxi0pookyEnBCds3Dh...	TECHNICIAN	0	2026-05-13 13:51:48	2026-05-13 13:51:48
2	8	la	lala	lala@test.com	0749999999	\$2b\$10\$HmkypsAmgK6yulr43eYq...	TECHNICIAN	0	2026-06-03 23:59:53	2026-06-03 23:59:53
3	9	cha	chacha	chacha@test.com	0749000000	\$2b\$10\$2O3q52cPtt4OemuNJR...	TECHNICIAN	1	2026-06-04 09:47:02	2026-06-23 13:13:54
4	10	DepanGO	Admin	admin@depan...	0600000000	\$2b\$10\$DmTdpRg3FW2QUM5iV...	ADMIN	1	2026-06-22 22:44:28	2026-06-22 22:44:28

FIGURE 17 – Aperçu de la base de données DepanGO dans MySQL Workbench

8 Développement du Back-end

8.1 Architecture générale du Back-end

Le back-end de DepanGO repose sur une architecture inspirée du modèle MVC, adaptée à une API REST. Les responsabilités sont réparties entre plusieurs composants spécialisés :

- Les **routes** reçoivent les requêtes HTTP et les dirigent vers le traitement approprié ;
- Les **contrôleurs** appliquent la logique métier ;
- Les **modèles Sequelize** représentent les données et communiquent avec MySQL ;
- Les **middlewares** contrôlent l'accès aux ressources protégées ;
- Les **utilitaires** contralisent certains traitements réutilisables, notamment l'envoi d'e-mails et la gestion des mots de passe.

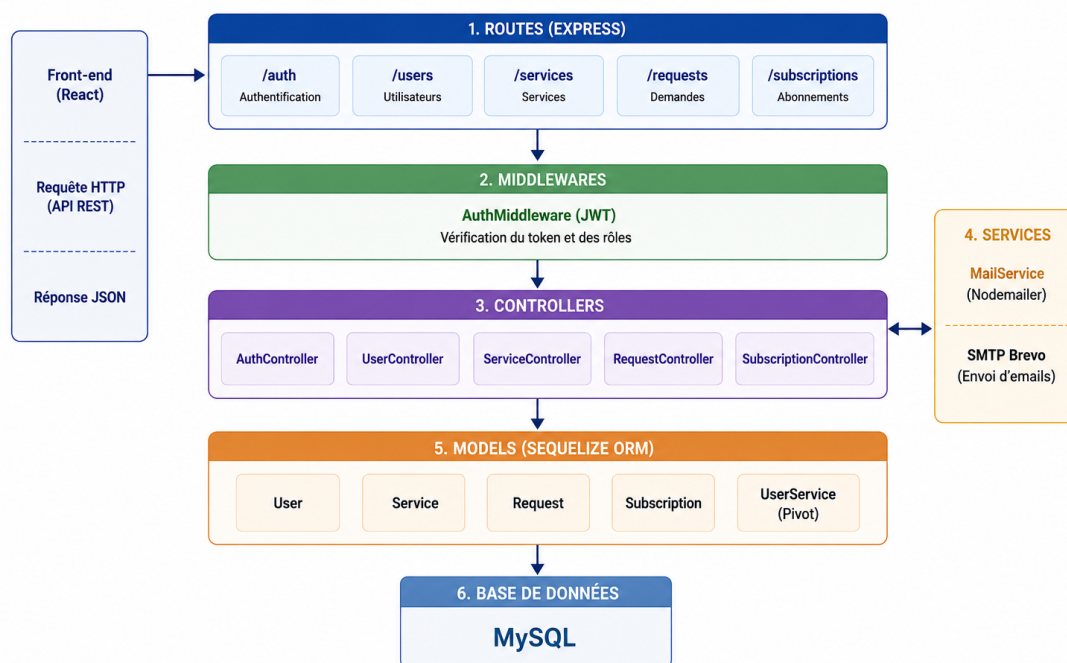


FIGURE 18 – Architecture générale du Back-end de DepanGO

La figure 18 présente le parcours général d'une requête. Le front-end React communique avec l'API Express. La route concernée transmet la requête au contrôleur, qui applique le traitement nécessaire et utilise les modèles Sequelize pour accéder aux données. Selon la fonctionnalité, un middleware peut intervenir pour vérifier l'authentification et les droits d'accès. Enfin, une réponse JSON est renvoyée au Front-end.

8.2 Fonctionnalité n°1 : Authentification sécurisée

L'authentification permet aux techniciens et aux administrateurs d'accéder aux espaces qui leur sont réservés.

Lorsqu'un utilisateur saisit son adresse e-mail et son mot de passe, le Back-end recherche son compte dans la base de données. Le mot de passe fourni est ensuite comparé au mot de passe haché enregistré. Si les identifiants sont valides, un jeton JWT d'une durée limitée est généré et renvoyé au Front-end.

8.2.1 Fonctionnement général

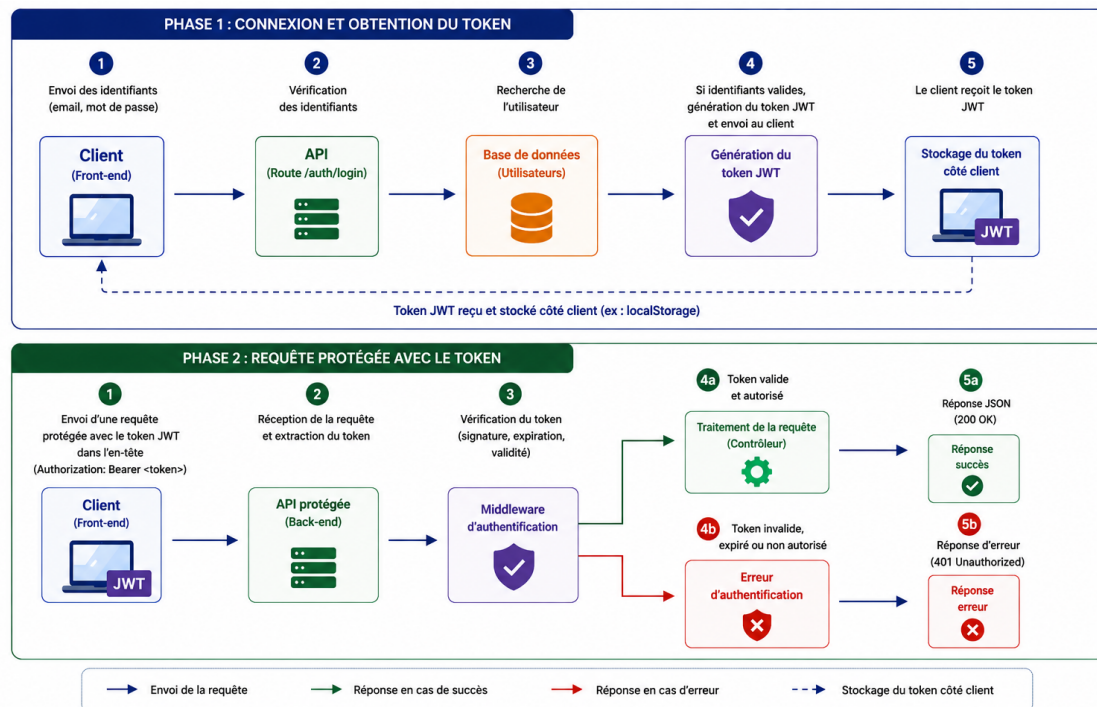


FIGURE 19 – Fonctionnement de l'authentification sécurisée avec JWT

La figure 19 présente le cycle complet de l'authentification. Après la connexion, le jeton retourné au Front-end accompagne les requêtes vers les ressources protégées. Le middleware vérifie alors sa validité, recherche l'utilisateur correspondant et contrôle son rôle avant d'autoriser ou de refuser l'accès.

Pour développer cette fonctionnalité, plusieurs technologies ont été utilisées :

- **Express** pour définir les routes d'authentification ;
- **Sequelize ORM** pour rechercher l'utilisateur dans la base de données ;
- **bcrypt**, utilisé à travers la fonction `comparePassword()`, pour comparer le mot de passe saisi avec le mot de passe haché enregistré ;
- **JWT** pour générer un jeton sécurisé ;
- **AuthMiddleware** pour contrôler les accès aux routes protégées.

8.2.2 Route d'authentification

La route `POST /auth/login` constitue le point d'entrée de l'authentification. Elle délègue le traitement au contrôleur `AuthController.login()`

```
import express from "express";
import AuthController from "../controllers/AuthController.js";

const authRouter = express.Router();

authRouter.post("/login", AuthController.login);

authRouter.post("/register", AuthController.register);

export default authRouter;
```

FIGURE 20 – Route d'authentification (Auth.route.js)

8.2.3 Vérification des identifiants

Le contrôleur recherche l'utilisateur dans la base de données grâce à Sequelize. Si celui-ci existe, le mot de passe est comparé à l'aide de bcrypt. Lorsque la vérification est réussie, un jeton JWT est généré avec une durée de validité par heure par défaut puis renvoyé au Front-end.

```
function login(req, res) {
  const { email, password } = req.body;

  // Sequelize
  User.findOne({ where: { email } }).then((user) => {
    if (!user) {
      return res.status(401).json({ error: "Email ou mot de passe incorrect" });
    }

    // Fonctions utilisateurs
    comparePassword(password, user.password).then((isMatch) => {
      if (!isMatch) {
        return res.status(401).json({ error: "Email ou mot de passe incorrect" });
      }

      // jwt librairie
      const token = jwt.sign({ email }, process.env.JWT_SECRET, {
        expiresIn: process.env.JWT_EXPIRES_IN || "1h",
      });

      // Express response
      return res.status(200).json({
        message: "Login successful",
        id: user.id,
        first_name: user.first_name,
        last_name: user.last_name,
        email: user.email,
        role: user.role,
        token,
      });
    });
  });
}
```

FIGURE 21 – Authentification de l'utilisateur (AuthController.js)

Le front-end stocke ensuite le jeton afin de l'utiliser lors de prochaines requêtes nécessitant une authentification.

Ce traitement montre l'utilisation de Sequelize dans une fonctionnalité réelle :

`User.findOne()` recherche l'utilisateur directement dans la base de données à partir de son e-mail. Le jeton retourné permet ensuite au Front-end de prouver l'identité de l'utilisateur lors des appels protégés.

8.2.4 Protection des routes

Les routes réservées aux techniciens et aux administrateurs sont protégées par un middleware chargé de vérifier la présence et la validité du jeton JWT avant d'autoriser l'accès aux ressources.

```

try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);

  if (!decoded?.email) {
    return res.status(401).json({ error: "Invalid Payload" });
  }

  const user = await User.findOne({
    where: { email: decoded.email },
  });

  if (!user || (roles.length && !roles.includes(user.role))) {
    return res.status(401).json({
      error:
        "Permission denied, you are not authorized to access this resource",
    });
  }

  req.user = user;
  return next();
} catch (error) {
  return res.status(401).json({ error: error.message });
}

```

FIGURE 22 – Vérification du jeton JWT (AuthMiddleware.js)

Le middleware contrôle successivement :

- la présence du jeton ;
- sa validité ;
- l'existence de l'utilisateur ;
- son rôle dans l'application.

Si toutes les vérifications sont validées, la requête est transmise au contrôleur correspondant.

Bilan de la fonctionnalité :

Cette fonctionnalité démontre l'utilisation conjointe d'**Express**, **Sequelize ORM**, **bcrypt**, **JWT** et des **middlewares** afin de mettre en oeuvre une authentification fiable et sécurisée. Elle constitue la base de la protection des différentes fonctionnalités réservées aux techniciens et aux administrateurs de DepanGO.

8.3 Fonctionnalité n°2 : Création d'une demande de dépannage

La création d'une demande constitue la fonctionnalité centrale de DepanGO. Elle permet à un client de transmettre son problème sans créer préalablement un compte.

Le client renseigne notamment ses coordonnées, son adresse, le service concerné et une description du problème. Après validation du formulaire côté Front-end, les données sont envoyées au Back-end puis enregistrées dans MySQL.

Pour mettre en oeuvre cette fonctionnalité, plusieurs technologies ont été utilisées :

- **React** pour développer le formulaire de création d'une demande ;
- **Axios** pour envoyer les données au back-end ;
- **Express** pour gérer les routes de l'API ;
- **RequestController** pour appliquer la logique métier ;
- **Sequelize ORM** pour manipuler les données dans MySQL.

8.3.1 Fonctionnement général

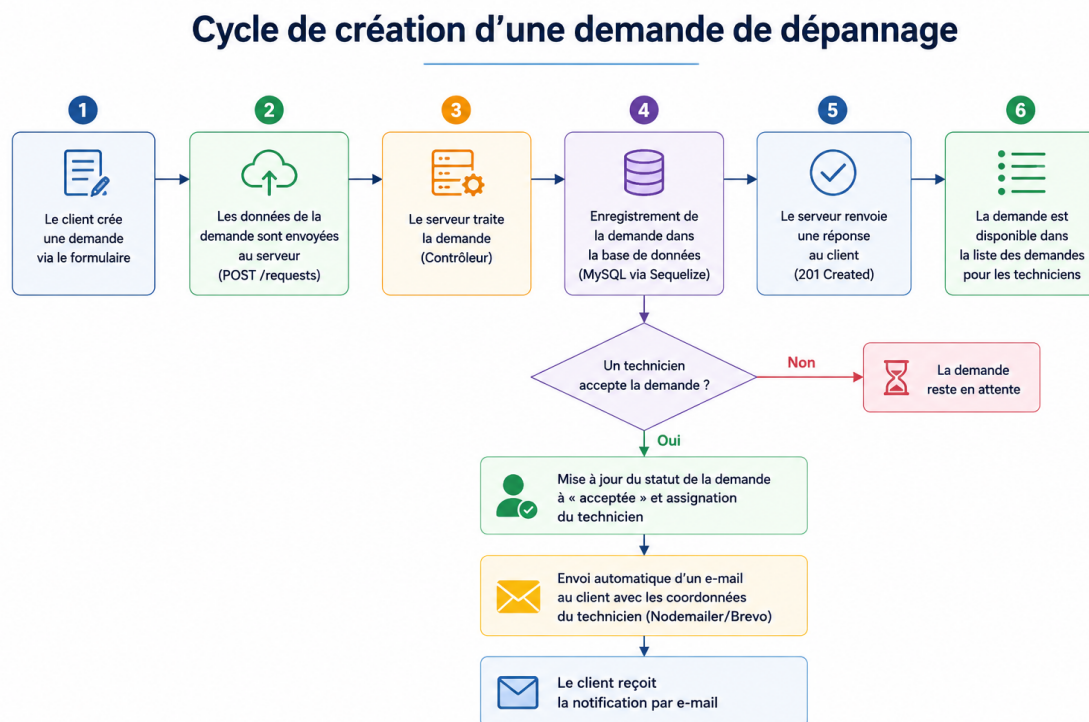


FIGURE 23 – Cycle de création d'une demande de dépannage

La Figure 23 présente le parcours de la demande depuis le client jusqu'à la base de données. Le formulaire React transmet les données à l'API. La route Express appelle ensuite `RequestController.createRequest()`, qui utilise le modèle Sequelize Request et pour créer l'enregistrement. Le serveur retourne enfin une réponse JSON avec le code HTTP 201.

Cette fonctionnalité montre concrètement la collaboration entre le Front-end, l'API REST, le contrôleur, Sequelize et MySQL.

8.3.2 Route de création d'une demande

La création d'une demande utilise la méthode HTTP POST.

```
// Créer une demande
requestRouter.post("/", RequestController.createRequest);

// Modifier une demande
requestRouter.put("/:id", RequestController.updateRequest);
```

FIGURE 24 – Route de création d'une demande (Request.route.js)

8.3.3 Traitement de la demande

Le traitement des informations est réalisé par la méthode `createRequest()` du `RequestController`. Après réception des données, le contrôleur crée une nouvelle demande en utilisant le modèle Request

de Sequelize. Une fois l'enregistrement effectué, une réponse HTTP 201 (Created) est renvoyée au Front-end accompagnée des informations de la nouvelle demande.

```
// Créer une demande
function createRequest(req, res) {
  Request.create(req.body).then((newRequest) => {
    res.status(201).json({
      message: "Demande créée",
      newRequest,
    });
  });
}
```

FIGURE 25 – Création d'une demande (RequestController.js)

8.3.4 Modèle de données utilisé

Afin de représenter une demande de dépannage, un modèle Request a été défini avec Sequelize. Il regroupe les principales informations nécessaires au traitement d'une intervention, notamment les coordonnées du client, son adresse, la description du problème, le statut de la demande ainsi que les dates liées à son évolution.

Le modèle est également associé aux modèles User et Service, ce qui permet de relier chaque demande au service concerné et lorsqu'elle est acceptée, au technicien chargé de l'intervention.

```
const Request = sequelize.define("Request", {
  description: {
    type: DataTypes.TEXT,
    allowNull: false,
  },
  last_name_client: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  first_name_client: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  adress_client: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  phone_client: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  email_client: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  status_request: {
    type: DataTypes.ENUM(
      "en_attente",
      "acceptee",
      "en_attente_paiement",
      "confirmee",
      "annulee"
    ),
    defaultValue: "en_attente",
  },
  acceptance_date: {
    type: DataTypes.DATE,
    allowNull: true,
  },
  confirmation_date: {
    type: DataTypes.DATE,
    allowNull: true,
  },
});
```

FIGURE 26 – Modèle Request (Request.js)

Bilan de la fonctionnalité :

Cette fonctionnalité met en évidence la collaboration entre plusieurs composants de l'application.

Le **Front-end React** permet au client de saisir les informations, **Axios** transmet ces données au serveur, **Express** réceptionne la requête, **RequestController** applique la logique métier, puis **Se-**

sequelize ORM enregistre les informations dans **MySQL** avant de retourner une réponse au Front-end.

8.4 Fonctionnalité n°3 : Acceptation d'une demande et notification automatique

Après sa création, une demande peut être consultée par un technicien correspondant au service concerné. Lorsqu'un professionnel accepte l'intervention, l'application doit mettre à jour la demande et informer automatiquement le client.

Cette fonctionnalité est particulièrement représentative de la logique métier de DepanGO, car une seule action du technicien déclenche plusieurs traitements successifs.

8.4.1 Fonctionnement général

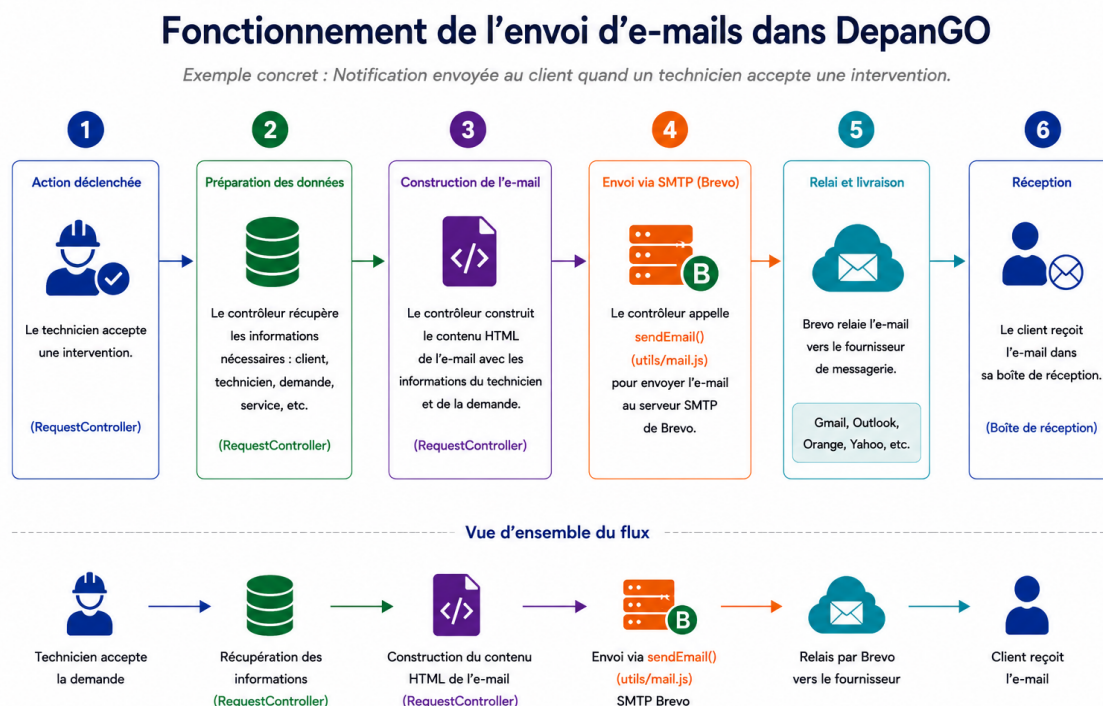


FIGURE 27 – Cycle d'acceptation d'une demande et de notification du client

Comme illustre la figure 27, lorsqu'un technicien clique sur le bouton **Accepter**, le Front-end envoie une requête HTTP PUT `/requests/:id` vers le Back-end. Le traitement suit alors plusieurs étapes :

1. **Réception de la requête** : la route Express reçoit la demande de mise à jour et la transmet à la méthode `updateRequest()` du `RequestController`.
2. **Recherche de la demande** : le contrôleur utilise l'identifiant présent dans l'URL pour retrouver la demande concernée.
3. **Vérification de son existence** : si aucune demande n'est trouvée, le serveur retourne une erreur HTTP 404.
4. **Mise à jour des informations** : les nouvelles données sont enregistrées avec Sequelize.
5. **Contrôle métier** : le Back-end vérifie que la demande possède le statut acceptée et qu'un technicien lui est associé.
6. **Notification du client** : les coordonnées du technicien sont récupérées puis transmises automatiquement au client par e-mail.

8.4.2 Mise à jour de la demande

Après réception de la requête, le contrôleur vérifie d'abord que la demande existe dans la base de données.

Lorsqu'elle est trouvée, ses informations sont mises à jour avec Sequelize. Selon les données transmises par le Front-end, cette opération permet notamment de modifier le statut de la demande et de lui associer un technicien.

```
// Modifier une demande
async function updateRequest(req, res) {
  const { id } = req.params;

  const request = await Request.findOne({
    where: { id },
  });

  if (!request) {
    return res.status(404).json({
      error: "Demande non trouvée",
    });
  }

  const updatedRequest = await request.update(req.body);
}
```

FIGURE 28 – Recherche et Mise à jour d'une demande (RequestController.js)

Le traitement repose sur plusieurs opérations successives.

`Request.findOne()` recherche d'abord la demande à partir de l'identifiant transmis dans l'URL. Si aucune ressource correspondante n'est trouvée, le serveur retourne une réponse HTTP 404.

Lorsque la demande existe, la méthode `request.update(req.body)` met à jour ses informations avec Sequelize.

8.4.3 Notification automatique du client

Une fois la demande mise à jour, le contrôleur vérifie que son nouveau statut correspond bien à une acceptation et qu'un technicien lui est associé. Cette condition métier évite de déclencher une notification lorsqu'une demande n'a pas réellement été prise en charge.

```
if (
  updatedRequest.status_request === "acceptee" &&
  updatedRequest.technician_id
) {
  const technician = await User.findPk(
    updatedRequest.technician_id
  );

  console.log("Technicien trouvé :", technician?.email);
  console.log("Email client :", updatedRequest.email_client);

  await sendEmail(
    updatedRequest.email_client,
    "Votre demande a été acceptée",
    `
    <h2>Votre demande a été acceptée</h2>

    <p>Un technicien a accepté votre demande.</p>

    <p><strong>Nom :</strong> ${technician.last_name}</p>
    <p><strong>Prénom :</strong> ${technician.first_name}</p>
    <p><strong>Email :</strong> ${technician.email}</p>
    <p><strong>Téléphone :</strong> ${technician.phone}</p>

    <br>

    <p>L'équipe DepanGO</p>
  `
  );
}

res.json(updatedRequest);
}
```

FIGURE 29 – Déclenchement de la notification automatique (RequestController.js)

La condition vérifie que `status_request` possède la valeur acceptée et qu'un `technicien_id` est associé à la demande. Lorsque ces deux conditions sont réunies, `User.findByPk()` récupère le professionnel concerné. Ses coordonnées sont ensuite intégrées au contenu HTML transmis à `sendEmail()`. Le client reçoit ainsi automatiquement le nom, le prénom, l'adresse e-mail et le numéro de téléphone du technicien ayant accepté l'intervention.

8.4.4 Envoi de l'e-mail avec Nodemailer et Brevo

Après la validation des conditions métier dans le `RequestController`, l'envoi de la notification est déclenché par l'appel à la fonction `sendEmail()`.

Dans l'implémentation de DepanGO, cette fonction est définie dans le fichier utilitaire `src/utils/mail.js`.

Elle s'appuie sur **Nodemailer** pour communiquer avec le serveur **SMTP de Brevo** et transmettre le message du client. Le processus technique est le suivant :

- le `RequestController` appelle la fonction `sendEmail()` ;
- l'adresse du client, l'objet du message et son contenu HTML sont transmis à la fonction ;
- Nodemailer utilise le transporteur SMTP configuré avec les paramètres de connexion ;
- Brevo assure ensuite l'acheminement du message vers la boîte de réception du client.

Cette organisation permet de centraliser la configuration de l'envoi des e-mails dans un fichier dédié, plutôt que de recréer une connexion SMTP directement dans chaque contrôleur.

```
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT,
  secure: false,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS,
  },
});

export async function sendEmail(to, subject, html) {
  return await transporter.sendMail({
    from: "DepanGO <chahinez.hatem@laplateforme.io>",
    to,
    subject,
    html,
  });
}
```

FIGURE 30 – Service d'envoi des e-mails (mail.js)

Le transporteur configure la communication avec le serveur SMTP. Les informations de connexion sont récupérées à partir des variables d'environnement `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER` et `SMTP_PASS`. Cette approche évite d'inscrire directement les identifiants SMTP sensibles dans le code source.

La fonction `sendEmail()` reçoit trois paramètres :

- **to** : l'adresse e-mail du destinataire ;
- **subject** : l'objet du message ;
- **html** : le contenu HTML de la notification.

Elle utilise ensuite `transporter.sendMail()` pour transmettre le message au serveur SMTP.

Cette séparation rend le code plus lisible et facilite la maintenance. La fonction d'envoi pourra également être réutilisée pour d'autres notifications de DepanGO sans dupliquer la configuration SMTP.

Bilan de la fonctionnalité :

Cette fonctionnalité illustre parfaitement la logique métier développée pour DepanGO.

A partir d'une simple action réalisée par le technicien, plusieurs traitements sont exécutés automatiquement :

- mise à jour des informations dans la base de données ;
- affectation du technicien à la demande ;
- changement du statut de l'intervention ;
- récupération des coordonnées du professionnel ;
- envoi automatique d'une notification au client.

Cette automatisation améliore considérablement la réactivité de l'application tout en limitant les interventions manuelles. Elle constitue l'une des fonctionnalités les plus importantes de DepanGO, puisqu'elle assure la mise en relation entre le client et le technicien.

8.5 Synthèse de l'API REST

Au cours du développement de DepanGO, plusieurs routes REST ont été mises en oeuvre afin de permettre la communication entre le Front-end et le Back-end Node.js

Ces routes couvrent les principales fonctionnalités de l'application, notamment l'authentification des utilisateurs, la gestion des profils, des services, des demandes de dépannage et des abonnements.

Le tableau suivant présente les principales routes utilisées :

Méthode	Route	Description
POST	/auth/login	Authentification d'un utilisateur
POST	/auth/register	Création d'un compte technicien
GET	/users/profile	Consultation du profil
PUT	/users/profile	Modification du profil
GET	/users/profile/services	Consultation des services du technicien
POST	/users/profile/services	Ajout d'un service
DELETE	/users/profile/services/:id	Suppression d'un service
GET	/services	Consultation des services
POST	/requests	Création d'une demande
PUT	/requests/:id	Mise à jour d'une demande
GET	/subscriptions	Consultation des abonnements

TABLE 23 – Les principales routes utilisées

L'organisation de ces routes s'appuie sur les principes d'une API REST, en utilisant les méthodes **HTTP GET, POST, PUT et DELETE** selon les opérations réalisées sur les ressources. Cette approche facilite les échanges entre le Front-end et le Back-end tout en conservant une organisation claire et cohérente.

9 Développement du Front-end

9.1 Architecture générale du Front-end

Le front-end de DepanGO est organisé en plusieurs dossiers spécialisés afin de séparer les responsabilités entre les pages, les composants réutilisables, les appels API et les structures communes de l'application.

9.1.1 Layouts et composants réutilisables

DepanGO utilise plusieurs layouts afin d'adapter la structure de l'interface au contexte d'utilisation :

- *PublicLayout* pour les pages publiques ;
- *TechnicianLayout* pour l'espace professionnel ;
- *AdminLayout* pour l'espace d'administration.

Les layouts regroupent les éléments communs à plusieurs pages et évitent leur duplication. Par exemple, l'espace technicien conserve une structure de navigation commune tandis que le contenu principal varie selon la page consultée.

Le projet s'appuie également sur des composants réutilisables tels que Navbar, Footer, Button ou EmergencyCTA. Cette approche améliore la cohérence visuelle de l'application et facilite la maintenance du code.

9.1.2 Navigation et routage

La navigation de DepanGO est gérée avec React Router. Les routes sont organisées selon les différents espaces de l'application.

Les pages publiques utilisent *PublicLayout*, tandis que l'espace technicien repose sur *TechnicianLayout* et est protégé par *RoleGuard*. Les routes imbriquées permettent ensuite d'afficher les différentes pages métier dans la structure correspondante.

```
<Routes>
  { /* Routes publiques */ }
  <Route path="/" element={ <PublicLayout /> } >
    <Route index element={ <Home /> } />
    <Route path="/auth/login" element={ <Login /> } />
    <Route path="/auth/register" element={ <Register /> } />
    <Route path="/services" element={ <Services /> } />
    <Route path="/how-it-works" element={ <HowItWorks /> } />
    <Route path="/become-technician" element={ <BecomeTechnician /> } />
    <Route path="/about" element={ <About /> } />
    <Route path="/request" element={ <CreateRequest /> } />
  </Route>

  { /* Routes privées */ }
  <Route
    path="technician"
    element={
      <RoleGuard allowedRoles={["TECHNICIAN"]} >
        <TechnicianLayout />
      </RoleGuard>
    }
  >
    <Route index element={ <TechnicianDashboard /> } />
    <Route path="requests" element={ <Requests /> } />
    <Route path="interventions" element={ <Interventions /> } />
    <Route
      path="completed-interventions"
      element={ <CompletedInterventions /> }
    />
    <Route path="subscription" element={ <Subscription /> } />
    <Route path="profile" element={ <Profile /> } />
    <Route path="profile/edit" element={ <EditProfile /> } />
  </Route>
</Routes>
```

FIGURE 31 – Organisation des routes publiques et protection de l'espace technicien (main.jsx)

Ce code montre deux organisations distinctes. Les pages publiques sont regroupées sous *PublicLayout*, tandis que l'espace technicien est accessible uniquement à un utilisateur possédant le rôle **TECHNICIAN**.

L'utilisation des routes imbriquées permet de conserver un layout commun tout en faisant varier la page affichée selon l'URL.

9.1.3 Communication avec l'API

Les appels HTTP vers le Back-end sont centralisés dans le dossier `api` et réalisés avec `Axios`. Cette séparation évite de répéter la configuration des requêtes directement dans chaque `React`.

Selon les fonctionnalités, `React Query` est également utilisé pour récupérer les données du serveur, gérer leur état et actualiser l'interface après certaines opérations.

Le fonctionnement général peut être résumé ainsi :

Utilisateur → page ou composant `React` → appels `Axios` → `API Express` → réponse `JSON` → mise à jour de l'interface.

Cette organisation constitue la base des trois fonctionnalités présentées dans les sections suivantes.

9.2 Fonctionnalité n°1 : Authentification et accès à l'espace technicien

L'authentification permet à un technicien enregistré d'accéder à son espace personnel. Cette fonctionnalité complète directement le mécanisme Back-end présenté dans le chapitre précédent.

Côté Front-end le processus consiste à :

1. récupérer les identifiants saisis ;
2. transmettre ces informations à l'API ;
3. récupérer les données retournées après authentification ;
4. conserver le jeton `JWT` ;
5. utiliser ce jeton pour les requêtes protégées ;
6. contrôler l'accès aux pages selon le rôle de l'utilisateur ;

9.2.1 Connexion de l'utilisateur

L'utilisateur renseigne son adresse e-mail et son mot de passe dans le formulaire de connexion. lors de la soumission, le Front-end transmet les identifiants au Back-end via l'endpoint :

`POST /auth/login`

Le parcours peut être résumé ainsi :

Formulaire de connexion → appel `API` → vérification Back-end → retour du `JWT` → accès à l'espace autorisé

La connexion est gérée avec une mutation. Lorsque l'authentification réussit, les informations retournées par le serveur sont enregistrées dans le `localStorage`, puis l'utilisateur est redirigé vers l'espace correspondant à son rôle.


```

const loginMutation = useMutation({
  mutationFn: async (data) => {
    return await login(data);
  },
  onSuccess: (response, variables, context) => {
    // If you are logged
    localStorage.setItem("user_id", response.data?.id);
    localStorage.setItem("first_name", response.data?.first_name);
    localStorage.setItem("last_name", response.data?.last_name);
    localStorage.setItem("email", response.data?.email);
    localStorage.setItem("role", response.data?.role);
    localStorage.setItem("token", response.data?.token);

    switch (response.data?.role) {
      case "ADMIN":
        navigate("/admin");
        break;

      case "TECHNICIAN":
        navigate("/technician");
        break;

      default:
        navigate("/");
        break;
    }
  },
  onError: (error) => {
    toast.error(
      error.response?.data?.error ||
      error.message ||
      "Une erreur est survenue",
    );
  },
});

function onSubmit(data) {
  return loginMutation.mutate(data);
}

return (

```

FIGURE 32 – Gestion de la connexion côté React (Login.jsx)

Ce traitement centralise plusieurs actions : appel de la fonction de connexion, stockage du JWT, conservation du rôle et redirection vers l'espace approprié.

En cas d'échec, un message d'erreur est affiché afin d'informer l'utilisateur.

9.2.2 Utilisation du JWT dans les requêtes Axios

Pour accéder aux ressources protégées, le jeton JWT doit être transmis au Back-end dans l'en-tête HTTP :

Authorization : Bearer <token>

Dans DepanGO, un intercepteur Axios récupère automatiquement le token enregistré dans le `localStorage` et l'ajoute aux requêtes.

```

// Intercepteur des requêtes
instance.interceptors.request.use(
  async (config) => {
    const token = localStorage.getItem("token");

    if (token !== null) {
      config.headers.Authorization = `Bearer ${token}`;
    }

    return config;
  },
  (error) => {
    console.log("une erreur est survenue:", error);
    return Promise.reject(error);
  },
);

```

FIGURE 33 – Ajout du jeton JWT aux requêtes Axios (config.js)

Cette configuration évite de répéter manuellement l'ajout du JWT dans chaque appel protégé.

Le Front-end transmet ainsi automatiquement le jeton au Back-end, où `AuthMiddleware` vérifie sa validité avant d'autoriser l'accès à la ressource demandée.

9.2.3 Contrôle de l'accès selon le rôle

DepanGO distingue principalement les rôles `TECHNICIEN` et `ADMIN`. Le composant `RoleGuard` vérifie si le rôle enregistré dans le `localStorage` appartient aux rôles autorisés pour l'espace demandé.

```
import { Navigate } from "react-router";

export function RoleGuard({ allowedRoles, children }) {
  const userRole = localStorage.getItem("role");

  if (allowedRoles.includes(userRole)) {
    return children;
  } else {
    // return <Navigate to="/login" />;
    return <Navigate to="/auth/login" replace />;
  }
}
```

FIGURE 34 – Protection d'une route selon le rôle (RoleGuard.jsx)

Lorsque le rôle est autorisé, le contenu demandé est affiché. Dans le cas contraire, l'utilisateur est redirigé vers la page de connexion.

Ce contrôle améliore la navigation côté Front-end et complète la vérification réalisée côté Back-end, qui reste indispensable pour sécuriser réellement les ressources de l'API.

Bilan de la fonctionnalité

L'authentification côté Front-end assure le lien entre le formulaire React et le mécanisme JWT du Back-end. Le jeton retourné après connexion est utilisé dans les requêtes protégées, tandis que le rôle permet d'adapter l'accès aux différents espaces de DepanGO.

9.3 Fonctionnalité n°2 : Création d'une demande côté client

La création d'une demande constitue le parcours principal de l'espace public. Le client peut transmettre son besoin de dépannage sans créer préalablement de compte.

le formulaire recueille notamment :

- le service concerné ;
- l'adresse du client ;
- la description du problème ;
- le nom et le prénom ;
- le numéro de téléphone ;
- l'adresse e-mail.

Le parcours peut être résumé ainsi ; Saisie des informations → validation → envoi à l'API → confirmation

9.3.1 Gestion du formulaire

La gestion du formulaire repose sur React Hook Form, tandis que Zod définit les règles de validation. Les deux outils sont reliés avec `zodResolver`.

```
// Schema de validation avec zod
const requestSchema = z.object({
  first_name_client: z.string().min(2),
  last_name_client: z.string().min(2),
  phone_client: z.string().min(10),
  email_client: z.string().email(),
  adress_client: z.string().min(5),
  description: z.string().min(10),
  service_id: z.string().min(1, "Veuillez choisir un service"),
});

// useForm avec zodResolver
export default function CreateRequest() {
  // Récupérer les paramètres de recherche de l'URL
  const [searchParams] = useSearchParams();

  const addressFromHero = searchParams.get("address") || "";
  const serviceIdFromHero = searchParams.get("service_id") || "";

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors },
  } = useForm({
    resolver: zodResolver(requestSchema),
    defaultValues: {
      adress_client: addressFromHero,
      service_id: serviceIdFromHero,
    },
  });
}
```

FIGURE 35 – Gestion et validation du formulaire de demande (CreateRequest.jsx)

Le schéma impose plusieurs contraintes avant l'envoi des données : longueur minimale de certains champs, format de l'adresse e-mail et sélection obligatoire d'un service.

React Hook Form centralise ensuite la gestion de la soumission et des erreurs. Les paramètres `address` et `service_id` peuvent également être récupérés depuis l'URL afin de préremplir le formulaire selon le parcours précédent du client.

9.3.2 Validation des données avec Zod

Avant l'envoi au back-end, les données saisies sont contrôlées afin de détecter certaines erreurs côté interface.

Zod permet de définir les règles de validation, par exemple :

- présence des champs obligatoires ;
- format attendu de l'adresse e-mail ;
- contraintes appliquées aux différentes valeurs du formulaire.

Lorsque les données sont valides, une mutation appelle la fonction `createRequest()`, qui transmet les informations au Back-end via l'endpoint : `POST / requests`

```

// useMutation pour créer une demande
const createRequestMutation = useMutation({
  mutationFn: async (data) => {
    return await createRequest(data);
  },

  onSuccess: (response) => {
    console.log("TOAST TEST");
    toast.success(response.data?.message);
    reset();
  },

  onError: (error) => {
    toast.error(
      error.response?.data?.error ||
      error.message ||
      "Une erreur est survenue",
    );
  },
});

// onSubmit
function onSubmit(data) {
  console.log(data);
  return createRequestMutation.mutate(data);
}

```

FIGURE 36 – Création de la demande et gestion du résultat (CreateRequest.jsx)

La fonction API appelée par cette mutation centralise l'envoi Axios :

```

async function createRequest(data) {
  return await instance.post("/requests", data);
}

```

En cas de succès, un message confirme la création de la demande et le formulaire est réinitialisé. En cas d'erreur, l'interface affiche un message adapté.

Bilan de la fonctionnalité

Cette fonctionnalité illustre le parcours complet d'une donnée côté Front-end : saisie par le client, validation, transmission à l'API et traitement du résultat. Elle complète directement la création avec `Request.create()` présentée dans le chapitre Back-end.

9.4 Fonctionnalité n°3 : Consultation et acceptation d'une demande côté technicien

L'espace technicien permet aux professionnels de consulter les demandes disponibles et d'interagir avec celles correspondant à leur activité.

Cette fonctionnalité complète le traitement Back-end présenté précédemment : côté React, le technicien consulte les demandes et déclenche l'action d'acceptation et côté serveur, la demande est mise à jour et le client est automatiquement informé.

9.4.1 Récupération des demandes avec React Query

La page technicien récupère deux ensembles de données :

- les demandes de dépannage ;
- les services associés au profil du technicien.

React Query est utilisé pour exécuter ces requêtes. Les données sont ensuite filtrées afin de conserver uniquement les demandes en attente correspondant aux services du professionnel.

```

// useQuery pour récupérer les demandes (charger les demandes disponibles pour le technicien)
const { data: requestsResponse } = useQuery({
  queryKey: ["requests"],
  queryFn: async () => {
    return await getRequests();
  },
});

const requests = requestsResponse?.data || [];

// useQuery pour récupérer les services du profil du technicien
const { data: profileServicesResponse } = useQuery({
  queryKey: ["profile-services"],
  queryFn: async () => {
    return await getProfileServices();
  },
});

const profileServices = profileServicesResponse?.data || [];

// Filtrer les demandes pour n'afficher que celles qui sont en attente et qui correspondent aux
// services du profil du technicien
const availableRequests = requests.filter((request) => {
  return (
    request.status_request === "en_attente" &&
    profileServices.some((service) => service.id === request.service_id)
  );
});

```

FIGURE 37 – Récupération et filtrage des demandes disponibles (Requests.jsx)

Ce traitement ne se limite pas à récupérer toutes les demandes. Il applique une logique de filtrage afin de présenter au technicien uniquement les interventions :

- possédant le statut `en_attente`
- correspondant à l'un des services associés à son profil.

Cette logique permet donc de personnaliser les demandes visibles selon les compétences du professionnel.

9.4.2 Affichage dynamique des demandes

Les demandes filtrées sont ensuite utilisées pour générer dynamiquement l'interface. Le technicien peut consulter les informations utiles à la prise de décision, notamment le service concerné, l'adresse et la description du problème.

L'affichage évolue en fonction des données retournées par l'API et ne repose donc pas sur un contenu statique.

9.4.3 Acceptation d'une demande

Lorsqu'un technicien clique sur le bouton Accepter, une mutation transmet au Back-end les informations nécessaires à la mise à jour :

- le nouveau statut `acceptee`;
- l'identifiant du technicien;
- la date d'acceptation.

L'appel API correspondant utilise l'endpoint : PUT /requests/:id

```
// useMutation pour accepter une demande avec le bouton "Accepter"
const acceptRequestMutation = useMutation({
  mutationFn: async (requestId) => {
    return await acceptRequest(requestId, {
      status_request: "acceptee",
      technician_id: Number(localStorage.getItem("user_id")),
      acceptance_date: new Date(),
    });
  },

  onSuccess: () => {
    queryClient.invalidateQueries({
      queryKey: ["requests"],
    });

    toast.success("Demande acceptée");
  },

  onError: (error) => {
    toast.error(
      error.response?.data?.error ||
      error.message ||
      "Une erreur est survenue",
    );
  },
});
```

FIGURE 38 – Acceptation d'une demande et actualisation de l'interface (Requests.jsx)

La mutation transmet les informations nécessaires à la mise à jour de la demande. Après, une réponse positive du serveur, `invalidateQueries()` : invalide les données associées à la clé `requests`, ce qui permet d'actualiser la liste affichée.

Un message confirme ensuite l'acceptation au technicien.

Cette action déclenche la logique métier étudiée côté Back-end :

- mise à jour de la demande avec Sequelize ;
- association du technicien ;
- contrôle du statut ;
- récupération des coordonnées du professionnel ;
- envoi automatique d'un e-mail au client.

Le Front-end et le Back-end interviennent ainsi de manière complémentaire dans une même fonctionnalité métier.

Bilan de la fonctionnalité

La consultation et l'acceptation d'une demande montrent comment une action réalisée dans l'interface React déclenche un traitement complet côté serveur. Le technicien agit depuis son espace personnel, l'API met à jour les données et le Front-end actualise l'affichage selon le résultat.

	A	B	C	D	E	F
	Test ID	Module	Description	Prérequis	Étapes	Résultat Attendu
1	AUTH001	Login	Se connecter avec email + mot de passe valides	Compte technicien existant	Aller à /auth/login → Saisir email et MDP corrects → Cliquer Login	Redirection vers /technician dashboard + localStorage remis à jour (user_id - first_name - last_name - email - role - token)
2	AUTH002	Login	Erreur login: email ou mot de passe incorrect (cas 1)	Email inexistant	Aller à /auth/login → Email inexistant → Cliquer Login	Toast d'erreur: message du backend (généralement "Email ou mot de passe incorrect")
3	AUTH003	Login	Erreur login: email ou mot de passe incorrect (cas 2)	Compte existant	Aller à /auth/login → Email correct + MDP faux → Cliquer Login	Toast d'erreur: message du backend (généralement "Email ou mot de passe incorrect")
4	AUTH004	Register	S'inscrire comme technicien	Aucun	Aller à /auth/register → Remplir formulaire complet (first_name - last_name - email - phone - password - confirm_password - role - service_id) → Cliquer Register	Toast succès: message backend + Redirection /auth/login
5	AUTH005	Register	Validation: first_name minimum 2 caractères	Aucun	Aller à /auth/register → first_name = "A" → Submit	Message d'erreur Zod (vérifier exact message backend)
6	AUTH006	Register	Validation: service_id obligatoire	Aucun	Aller à /auth/register → Ne pas sélectionner service → Submit	Message d'erreur "Veuillez choisir une spécialité"
7	AUTH007	Register	Validation: email format invalide	Aucun	Aller à /auth/register → Email invalide (ex: "test@") → Submit	Message d'erreur "Veuillez saisir une adresse email valide"
8	AUTH008	Register	Validation: téléphone minimum 10 caractères	Aucun	Aller à /auth/register → phone = "123456789" (9 chars) → Submit	Message d'erreur Zod
9	AUTH009	Register	Validation: mot de passe minimum 6 caractères	Aucun	Aller à /auth/register → password = "12345" → Submit	Message d'erreur "Le mot de passe doit contenir au moins 6 caractères"
10	AUTH010	Register	Validation: MDP et confirmation doivent correspondre	Aucun	Aller à /auth/register → password ≠ confirm_password → Submit	Message d'erreur "Les mots de passe ne correspondent pas"
11	AUTH011	Register	Message redirection si déjà connecté	Utilisateur avec email en localStorage	Aller à /auth/register	Message "You are already logged in as [email]" + Lien vers Home
12	AUTH012	Register	Email déjà utilisé (validation backend)	Email existant en base	Aller à /auth/register → Email déjà utilisé → Submit	Toast d'erreur: message backend (ex: "Email déjà utilisé")
13	JWT001	JWT Token	Token JWT stocké dans localStorage après login	Login réussi	Effectuer login → Ouvrir DevTools → Vérifier localStorage	clé "token" présente avec valeur Bearer token
14	JWT002	JWT Token	Token JWT envoyé en header Authorization	Utilisateur connecté	Effectuer action API (ex: accepter demande) → Ouvrir Network tab → Vérifier requête	Header "Authorization: Bearer [token]" présent dans requête
15	JWT003	JWT Token Expiré	Si token expiré (401): redirection + alert + localStorage vidé	Token expiré en localStorage	Effectuer action / Server retourne 401	Alert "Votre session a expiré" + Redirection /auth/login + localStorage complètement vidé (token - email - role - user_id - first_name - last_name)
16	JWT004	JWT Rôle	Technicien ne peut pas accéder /admin	Technicien connecté (role=TECHNICIAN)	Accéder à /admin (URL directe ou manipulation)	Redirection /auth/login (RoleGuard bloque)
17	JWT005	JWT Rôle	Admin ne peut pas accéder /technician	Admin connecté (role=ADMIN)	Accéder à /technician (URL directe)	Redirection /auth/login (RoleGuard bloque)
18	PUB001	Accueil	Page d'accueil affiche sections principales	Aucun	Aller à /	Sections visibles: HeroSection - ServiceSection - EmergencyCTA - HowItWork
19	PUB002	Accueil	Bouton ScrollToTop visible et fonctionne	Aucun	Accéder à / → Scroller vers bas	Bouton visible → Clic remonte page en haut
20	PUB003	Navigation	Navigation: Accueil → Services	Aucun	Cliquer "Services" dans Navbar	Redirection vers /services

FIGURE 40 – Extrait du cahier de tests fonctionnels

10.2 Sécurité

10.2.1 Hashage des mots de passe

Les mots de passe sont hachés avec **bcrypt** avant d'être enregistrés dans la base de données. Ils ne sont donc jamais stockés en clair, ce qui renforce la sécurité des comptes utilisateurs.

La figure 41 présente la fonction `hashPassword()` utilisée pour réaliser ce hachage.

```

async function hashPassword(password) {
  return bcrypt.hash(password, SALT_ROUNDS);
}

```

FIGURE 41 – Hachage des mots de passe avec bcrypt (password.js)

10.2.2 Authentification par JWT

L'authentification utilise JSON Web Token (JWT). Après la connexion, un jeton est généré puis transmis au Front-end. Ce jeton est ensuite envoyé dans l'en-tête **Authorization** lors des requêtes nécessitant une authentification.

Ce fonctionnement est présenté au chapitre 8 :

- **Figure 19** : fonctionnement de l'authentification sécurisée avec JWT ;
- **Figure 20** : route d'authentification (`Auth.route.js`) ;
- **Figure 21** : authentification de l'utilisateur (`AuthController.js`) ;
- **Figure 22** : vérification du jeton par `AuthMiddleware`

10.2.3 Contrôle des accès

L'accès aux fonctionnalités est limité selon le rôle de l'utilisateur (**ADMIN** ou **TECHNICIEN**).

Le Back-end contrôle les routes avec **AuthMiddleware** et le Front-end protège les pages grâce à **RoleGuard**.

Ces mécanismes sont illustrés :

- **Figure 22** (chapitre 8) : **AuthMiddleware** ;
- **Figure 34** (chapitre 9) : **RoleGuard**.

10.2.4 Protection contre les injections SQL

Les accès à la base de données sont réalisés avec **Sequelize ORM**, qui génère des requêtes SQL paramétrées. Cette approche contribue à limiter les risques d'injection SQL.

Ce mécanisme est illustré :

- **Figure 21** : recherche d'un utilisateur avec **Sequelize**
- **Figure 25** : création d'une demande avec **Sequelize**.

10.2.5 Validation des données

Les informations saisies par les utilisateurs sont validées avec **Zod** avant leur envoi au Back-end. Cette validation permet de limiter les erreurs et les données invalides.

Ce mécanisme est présenté au **chapitre 9, Figure 36 : Validation des données avec Zod**

10.2.6 Configuration CORS

Une configuration **CORS** autorise les échanges entre le Front-end et le Back-end. Pendant le développement, toutes les origines sont autorisées afin de faciliter les tests. En production, cette configuration sera limitée aux domaines autorisés afin de renforcer la sécurité des communications entre les différents composants de l'application.

La figure 42 présente cette configuration dans le fichier principal de l'application.

```
configDotenv(); // Charger les variables d'environnement depuis le fichier .env

const app = express(); // Créer une application Express

app.use(cors({ origin: "*" })); // Autoriser les requêtes CORS de toutes origines
app.use(express.json());

const PORT = process.env.PORT || 3000; // Définir le port du serveur
```

FIGURE 42 – Configuration CORS de l'application (index.js)

10.2.7 Respect du RGPD

DepanGO applique le principe de minimisation des données en ne collectant que les informations strictement nécessaires au fonctionnement de l'application, telles que le nom, prénom, l'adresse e-mail et le numéro de téléphone. Les mots de passe sont hachés avec **bcrypt** et ne sont jamais enregistrés en clair, contribuant ainsi à la protection des données personnelles.

11 Déploiement

Le déploiement local permet d'exécuter l'application DepanGO sur un poste de développement afin de vérifier son bon fonctionnement avant une éventuelle mise en production. Cette étape consiste à préparer l'environnement, installer les dépendances, configurer les variables d'environnement, lancer les différents services et vérifier la communication entre le Front-end, le Back-end et la base de données.

11.1 Installation des dépendances

Avant de lancer DepanGO, les dépendances du Front-end et du Back-end doivent être installées.

Depuis la racine du projet, la commande suivante est exécutée : `npm run install`

Cette commande exécute successivement les commandes suivantes :

```
cd back && npm install
```

```
cd ../front && npm install
```

Les commandes `npm install` installent automatiquement toutes les dépendances définies dans les fichiers `package.json` du Back-end et du Front-end.

Pour le **Back-end**, elles installent notamment **Express.js**, **Sequelize**, **MySQL2**, **brypt**, **jsonwebtoken (JWT)**, **cors**, **dotenv** et **Nodemailer**. Ces bibliothèques permettent de développer l'API REST, de gérer la base de données, l'authentification, la sécurité des échanges et l'envoi d'e-mails.

Pour le **Front-end**, elles installent notamment **React**, **Vite**, **React Router**, **Axios**, **React Query**, **React Hook Form**, **Zod**, **Tailwind CSS** et **React Hot Toast**. Ces bibliothèques sont utilisées pour développer l'interface utilisateur, gérer la navigation, communiquer avec l'API REST, valider les formulaires, gérer les données et appliquer le style de l'application.

11.2 Configuration des variables d'environnement

Avant de démarrer l'application, le fichier `.env` situé dans le dossier `back` est configuré.

Ce fichier contient les variables d'environnement nécessaires au fonctionnement du Back-end, notamment le port du serveur, les paramètres de connexion à la base de données MySQL, la clé secrète utilisée pour l'authentification JWT ainsi que les informations permettant l'envoi d'e-mails via Brevo.

L'utilisation de fichiers `.env` permet de séparer les informations sensibles du code source et de faciliter la configuration de l'application selon l'environnement d'exécution.

11.3 Création de la base de données

Avant de lancer l'application, une base de données nommée **DepanGO** est créée dans **MySQL**.

Au démarrage du Back-end, **Sequelize ORM** établit automatiquement la connexion avec le serveur MySQL (port **3306**) et crée les tables de la base de données à partir des modèles définis dans l'application lorsqu'elles n'existent pas encore. Les relations entre les différentes tables sont également mises en place lors de cette initialisation.

L'outil **MySQL Workbench** est ensuite utilisé pour visualiser la structure de la base de données et vérifier les données enregistrées.

11.4 Lancement du Back-end

Le back-end est lancé depuis le dossier **back** avec la commande :

```
npm start
```

Cette commande exécute le script **start** défini dans le fichier **package.json**, qui lance **Nodemon** (**nodemon index.js**). Nodemon démarre le serveur **Express.js** et redémarre automatiquement l'application à chaque modification du code source.

Le serveur est ensuite accessible à l'adresse :

http ://localhost :3000

Une fois démarré, le Back-end expose les différentes routes de l'API REST, établit la connexion avec la base de données et traite les requêtes envoyées par le Front-end.

11.5 Lancement du Front-end

Le Front-end est lancé depuis le dossier **front** avec l'une des commandes suivantes :

```
npm start
```

ou

```
npm run dev
```

Dans ce projet, ces deux commandes exécutent respectivement les scripts **start** et **dev** définis dans le fichier **package.json**. Ces deux scripts lancent le serveur de développement **Vite**, qui compile automatiquement l'application React et recharge les pages à chaque modification du code source.

L'application est alors accessible à l'adresse suivante :

http ://localhost :5173

Le Front-end communique avec le Back-end afin d'envoyer les requêtes API et d'afficher les données aux utilisateurs.

11.6 Vérification du fonctionnement

Une fois les deux serveurs démarrés, l'application est accessible depuis un navigateur Web.

Le navigateur communique avec le Front-end développé en React. Celui-ci échange avec le Back-end **Node.js/Express.js** via l'API REST, qui traite les requêtes et interagit avec la base de données MySQL grâce à **Sequelize ORM**.

Cette architecture permet d'assurer une séparation claire entre l'interface utilisateur, la logique métier et le stockage des données.

11.7 Architecture du déploiement local

La figure 43 présente l'architecture locale de DepanGO ainsi que les principales étapes du déploiement. Elle illustre les échanges entre le navigateur, le Front-end React, le Back-end Node.js/Express.js et la base de données MySQL, ainsi que les différentes opérations nécessaires au lancement de l'application.

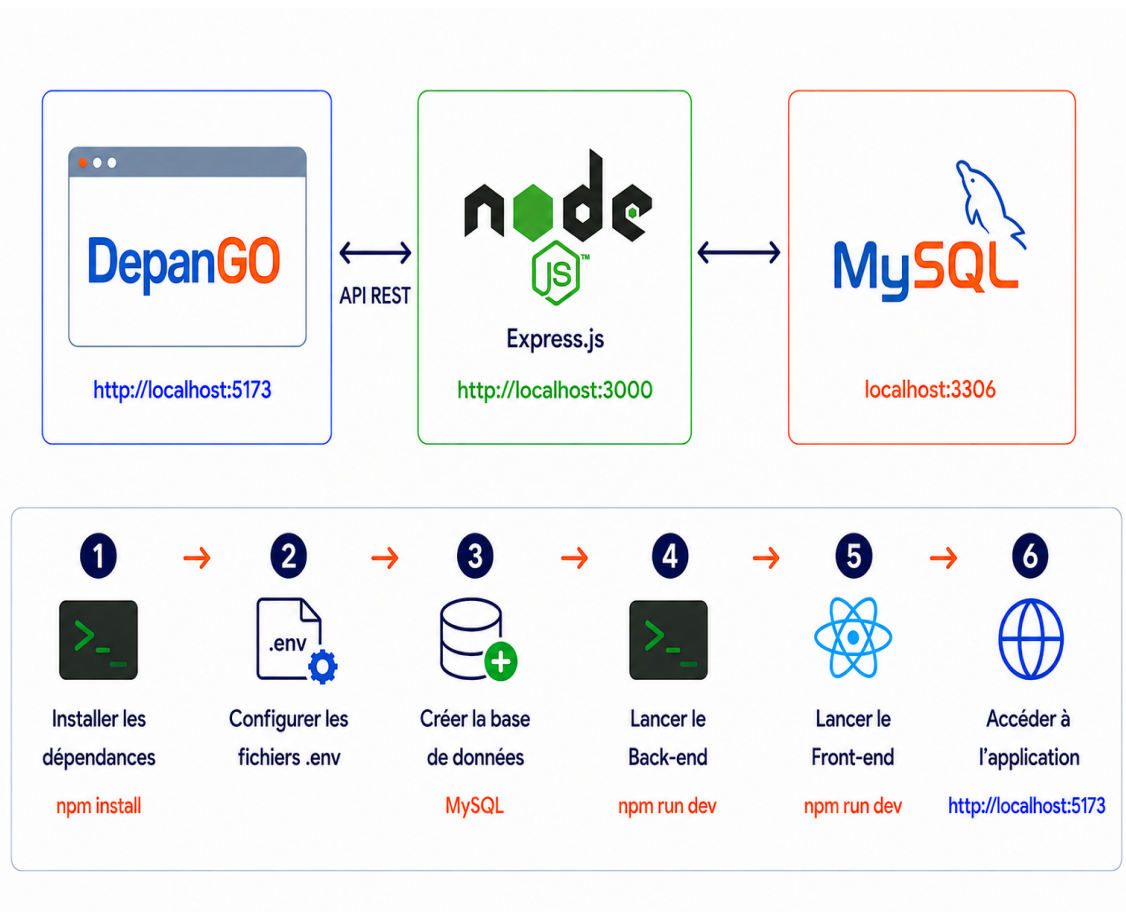


FIGURE 43 – Architecture locale et étapes du déploiement de l'application DepanGO

Conclusion générale

La réalisation de **DepanGO** m'a permis de mettre en pratique les compétences acquises au cours de la formation **Développeur Web et Web Mobile**. Ce projet m'a conduit à concevoir et développer une application Web complète, depuis l'analyse des besoins jusqu'au déploiement de l'application, en passant par la conception de la base de données, le développement du Front-end et du Back-end, la mise en oeuvre de l'API REST, la sécurisation des échanges et la réalisation des tests.

Le développement de DepanGO m'a également permis d'approfondir l'utilisation de technologies telles que **React**, **Node.js**, **Express.js**, **Sequelize ORM** et **MySQL**, ainsi que d'appliquer de bonnes pratiques en matière d'organisation du code, de sécurité et de gestion de projet.

Bien que les principales fonctionnalités prévues pour le MVP aient été réalisées, plusieurs évolutions pourront être envisagées, comme l'intégration d'un système de paiement en ligne, la géolocalisation des techniciens, les notifications en temps réel ou encore le déploiement de l'application sur un serveur de production.

Au-delà des aspects techniques, ce projet a été une expérience particulièrement enrichissante. Il m'a permis de développer mon autonomie, ma rigueur ainsi que ma capacité à faire face aux difficultés rencontrées tout au long du développement. Il représente une étape importante dans mon parcours professionnel et constitue une base solide pour poursuivre le développement de mes compétences et atteindre mon objectif de devenir développeuse full-stack.